

piep

inovação
em engenharia
de polímeros

CONVERTING
IDEAS INTO
SUSTAINABLE
PRODUCTS

www.piep.pt

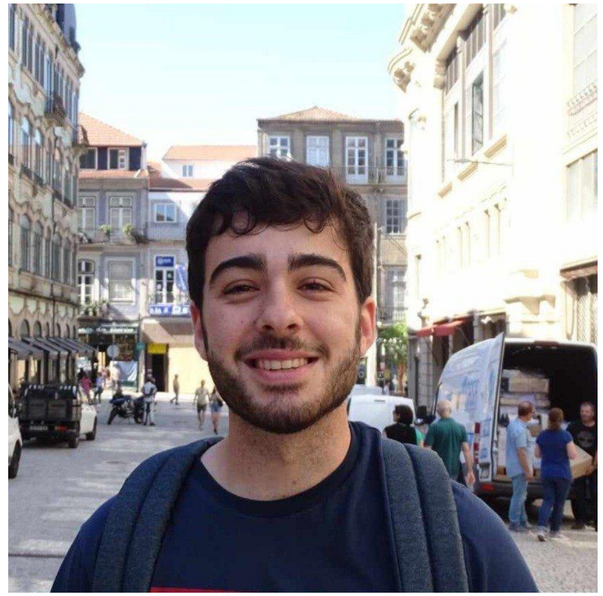
Formação GSS

Formador	Hugo Coutinho Gomes	Local de Formação	PIEP

Entidade Formadora Certificada dgert

Apresentação

- **Licenciatura e Mestrado em Engenharia Física pela Universidade de Aveiro**
- **No PIEP desde 09/2023**
- **Experiência profissional em desenvolvimento de software, programação, IA e eletrónica.**



Hugo Coutinho Gomes

hugo.gomes@piep.pt

Definição de um sistema de IA

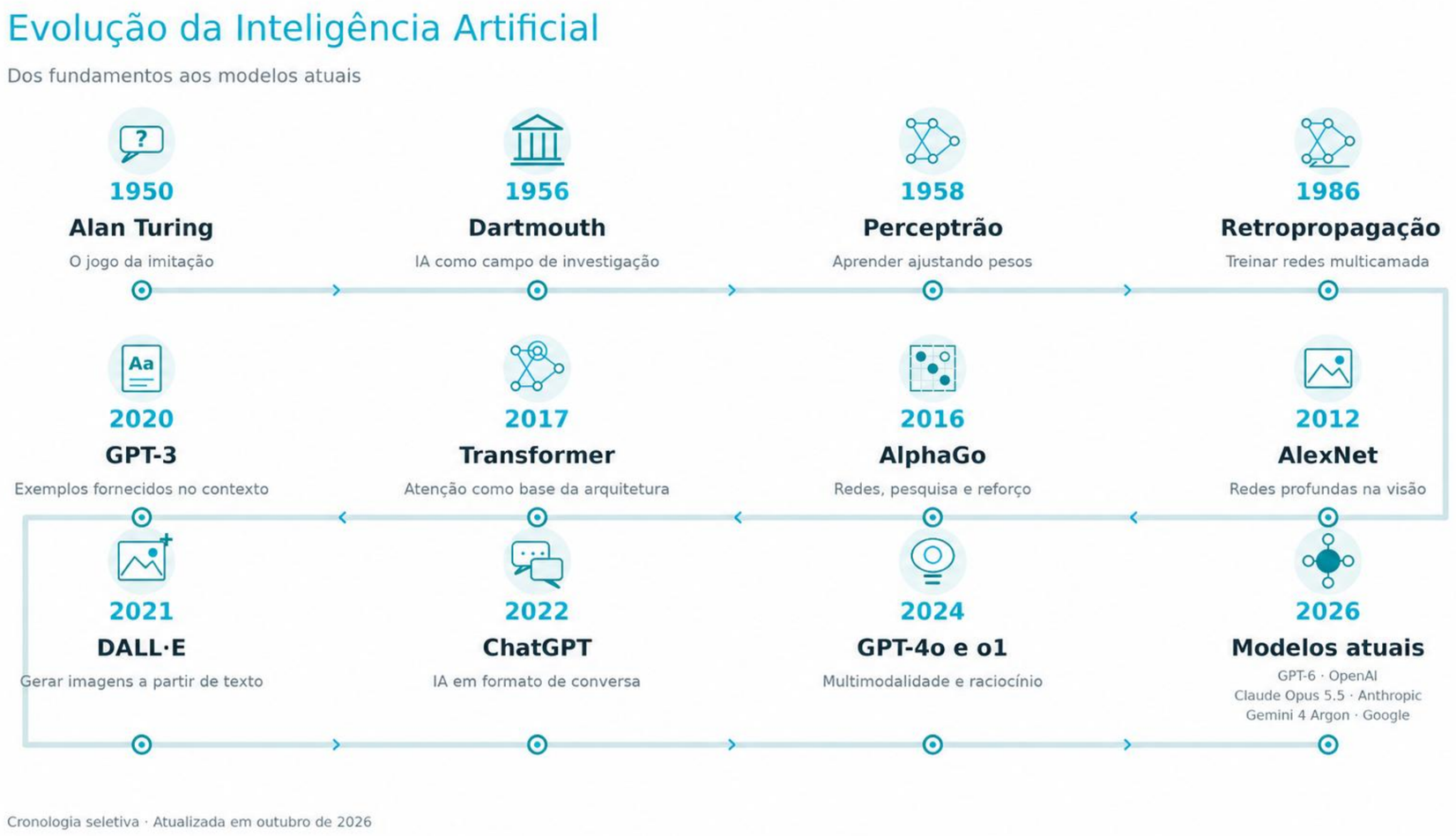
Um sistema de IA é um sistema computacional que utiliza informação de entrada para inferir como produzir previsões, conteúdos, recomendações ou decisões capazes de influenciar ambientes físicos ou virtuais.

Opera segundo objetivos explícitos ou implícitos, com autonomia variável. Alguns sistemas podem adaptar-se após a sua implementação.

OCDE

Organização para a Cooperação e Desenvolvimento Económico

Marcos relevantes para esta formação





Inteligência Artificial

Métodos

IA simbólica

Regras
e lógica

Aprendizagem automática

Aprender com dados

Aprendizagem profunda

Redes neuronais profundas

AGI e ASI: critérios em debate

Capacidades e organização

IA generativa

Criar conteúdo

IA com agentes

Escolher passos e usar ferramentas

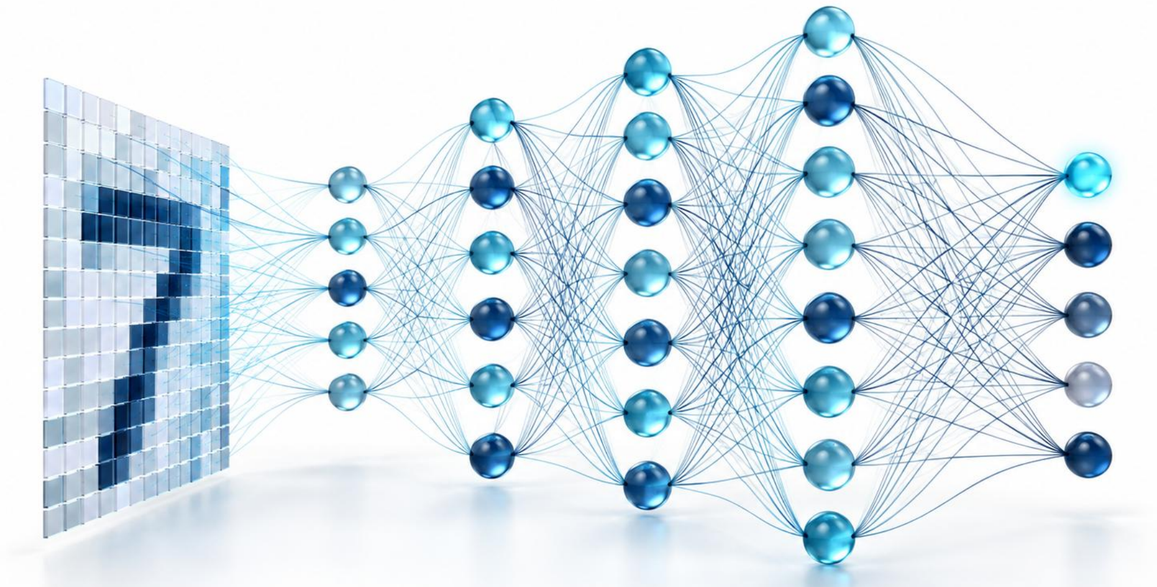
Conceitos de capacidade

AGI
IA geral

ASI
Superinteligência

Módulo 1

Aprendizagem automática e redes neuronais



Objetivos do módulo 1

Aprendizagem automática e redes neuronais

Distinguir aprendizagem supervisionada, não supervisionada e por reforço.

Relacionar características e redes neuronais com a classificação de dígitos.

Explicar a passagem direta, a perda e a retropropagação, distinguindo treino de inferência.

Interpretar métricas de avaliação e reconhecer sobreajustamento e generalização.

Distinguir tarefas de visão computacional e observar modelos pré-treinados em imagem, vídeo ou câmara.

Aprendizagem supervisionada

Aprende com exemplos cuja resposta já é conhecida.



Com respostas conhecidas

Cada sessão inclui o resultado: comprou ou não comprou.

Regressão logística

Usa esses exemplos para prever o resultado de uma nova sessão.

Dados, características e rótulos



Sessão	Páginas	Saída	Comprou?
A	2	65%	Não
B	6	25%	Sim
C	3	60%	Não

Dados: Sessão

Características: páginas e taxa de saída

Rótulo: comprou / não comprou

Treino e inferência



Treino

Comparar a previsão com o rótulo e ajustar os parâmetros.

Inferência

Prever a resposta de uma nova sessão com os parâmetros fixos.

Classificação e regressão

Ambas usam respostas conhecidas. A diferença está no tipo de saída.



Uma categoria

Regressão logística:
«comprou / não comprou».

Um valor numérico

Prever o montante gasto.
Exemplo: 30 €.

Aprendizagem não supervisionada

Procura padrões nos dados, sem respostas dadas para a tarefa.



Sem respostas dadas

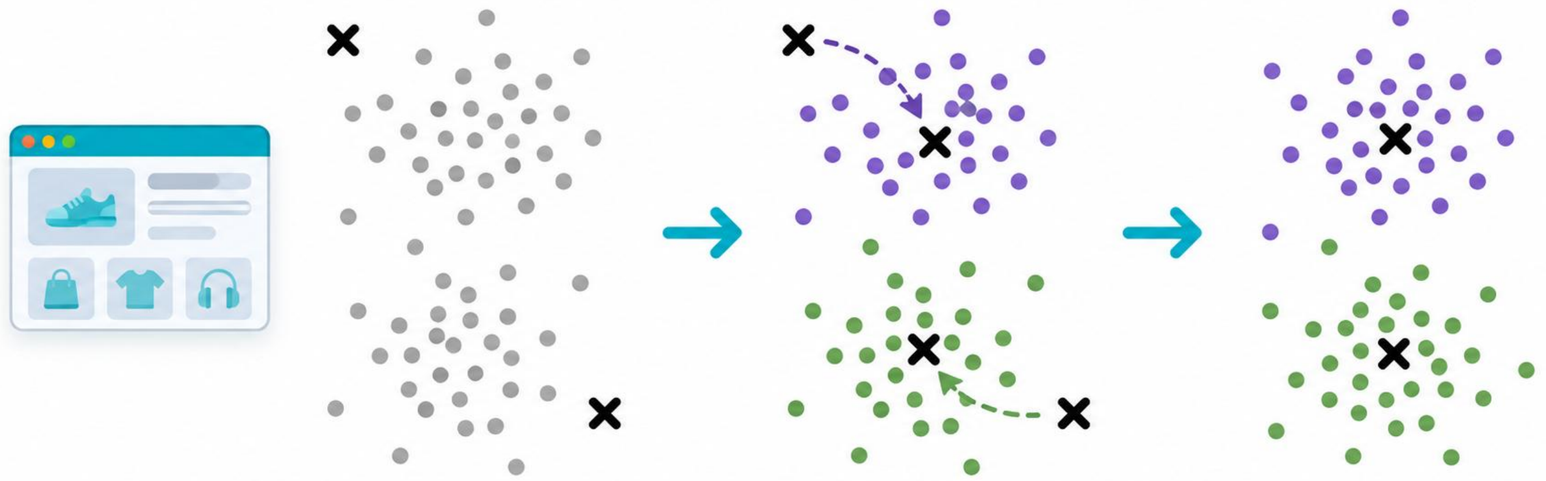
O modelo agrupa sessões com características semelhantes.

K-means

Os grupos encontrados não significam «comprou» e «não comprou».

K-means: descobrir grupos

Retiramos os rótulos de compra e agrupamos as sessões pelos seus atributos.



Processo

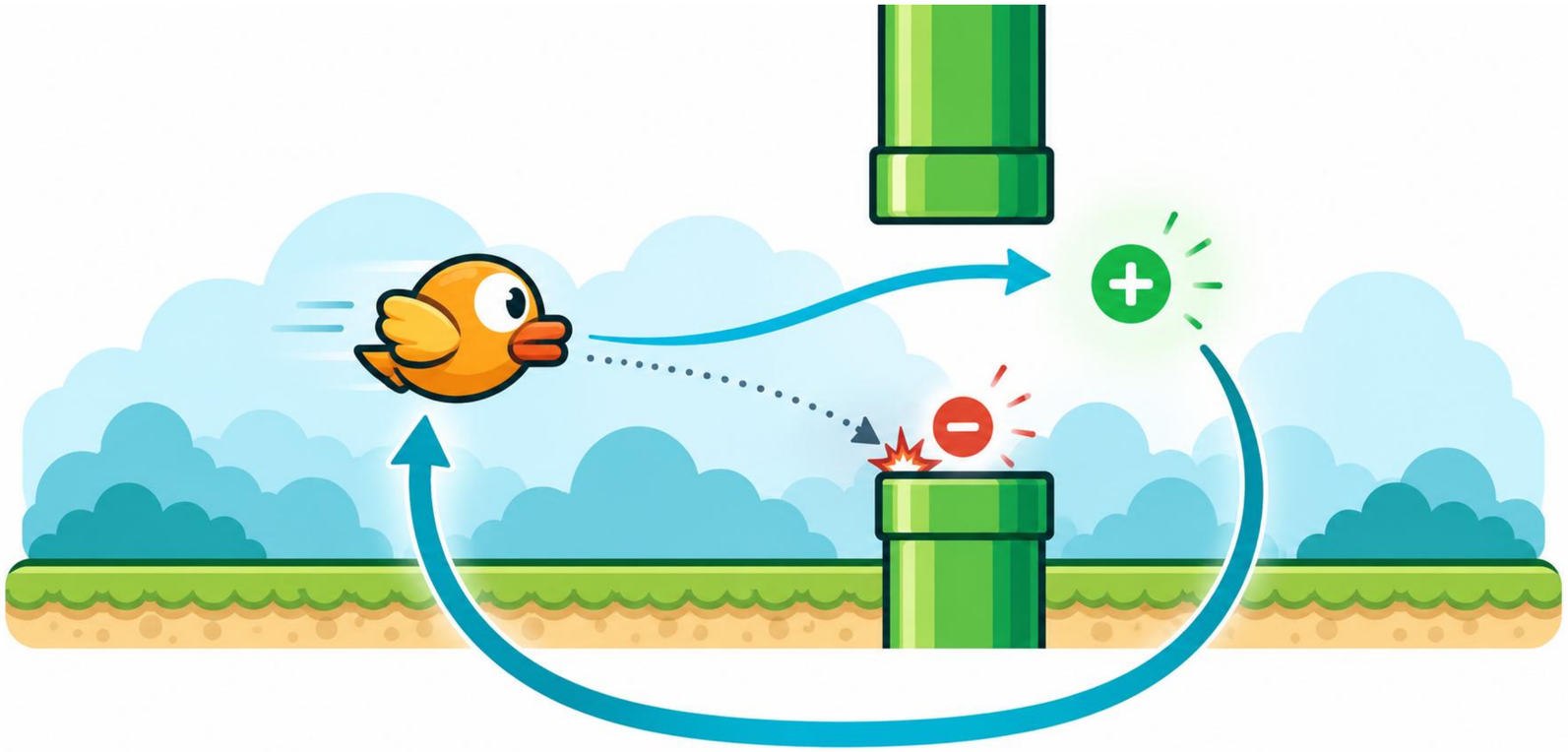
Escolher K, atribuir sessões ao centro mais próximo e ajustar os centros.

Interpretar os grupos

Os grupos descrevem semelhanças. O seu significado exige análise.

Aprendizagem por reforço

Aprende a escolher ações através da interação e das recompensas.



Ações e recompensas

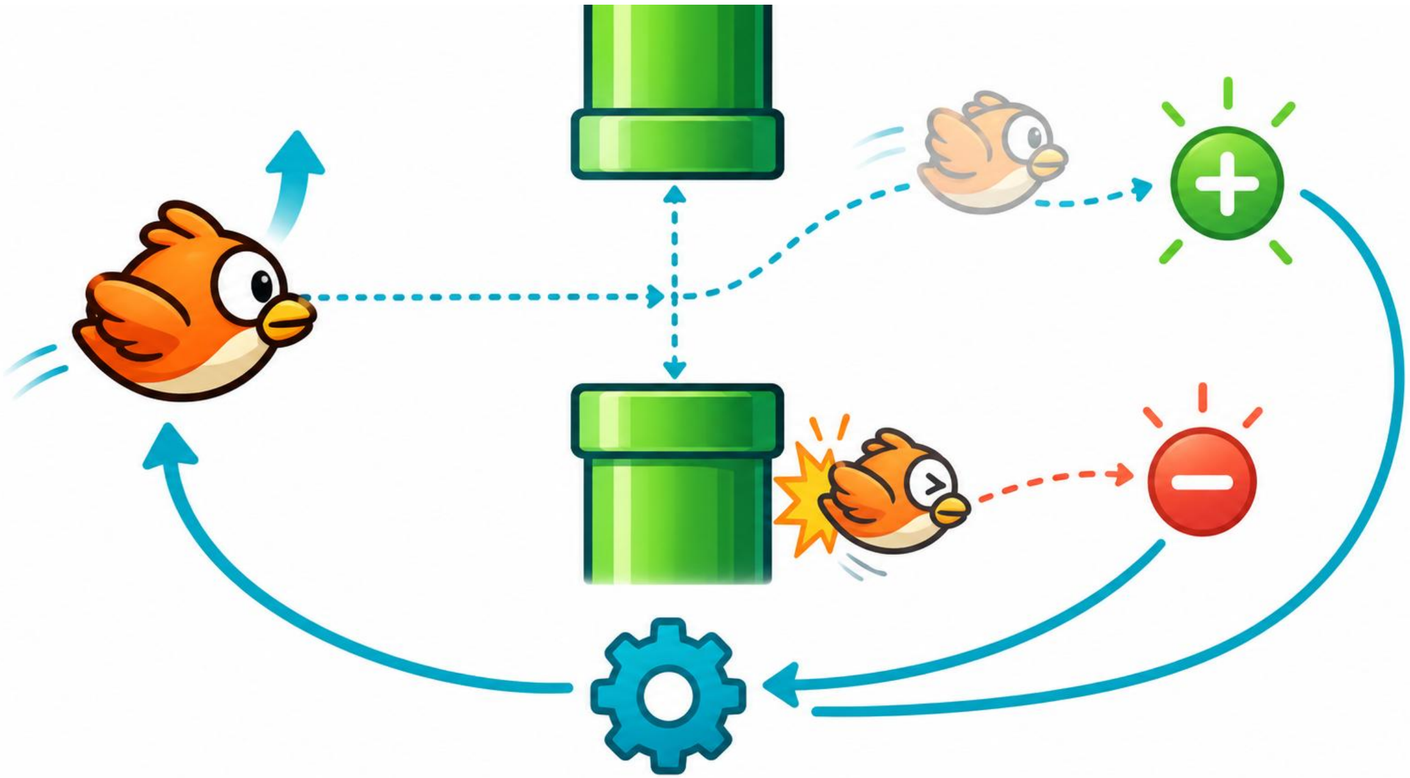
Saltar ou não saltar? A ação altera o estado do jogo.

Flappy Bird · Q-learning

Passar o tubo dá recompensa. Colidir dá penalização.

Flappy Bird: ações e recompensas

A política escolhe ações; o Q-learning ajusta-a com a resposta do jogo.



Observação

Distância ao tubo, desvio vertical e velocidade.

Ações

Saltar / não saltar.

Recompensas

Sobreviver: +0,1
Passar um tubo: +5
Colidir: −10

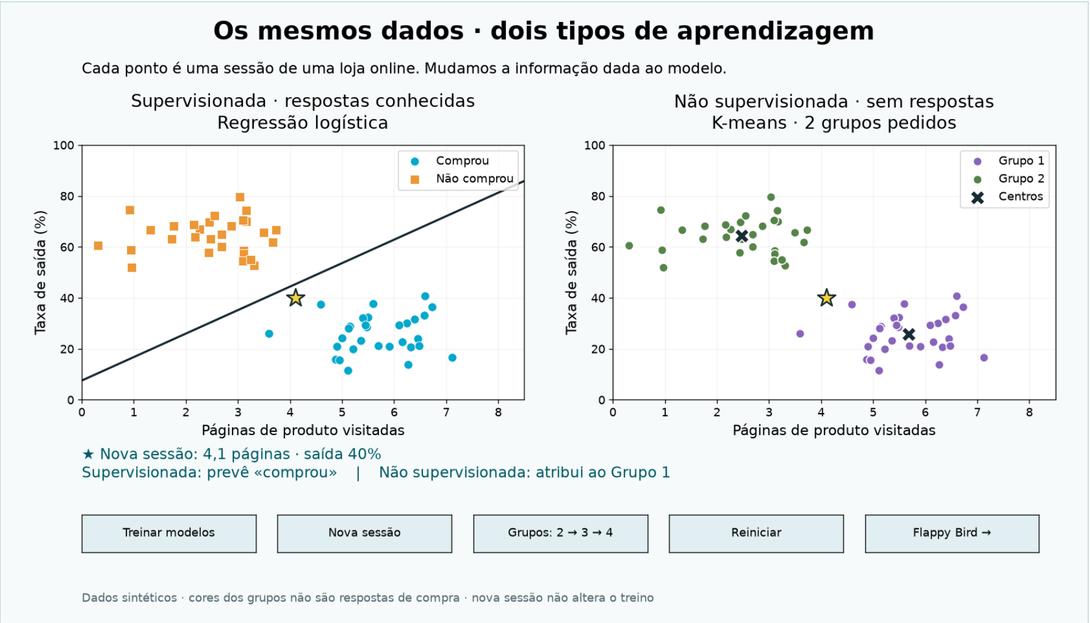
Demonstração: tipos de aprendizagem

Demonstração · exercício 01

```
python 01_paradigmas_aprendizagem.py
```

O que observar

- A mesma sessão: prever uma compra ou atribuir um grupo.
- Mudar o número de grupos e observar o resultado.
- No Flappy Bird, seguir ações e recompensas.

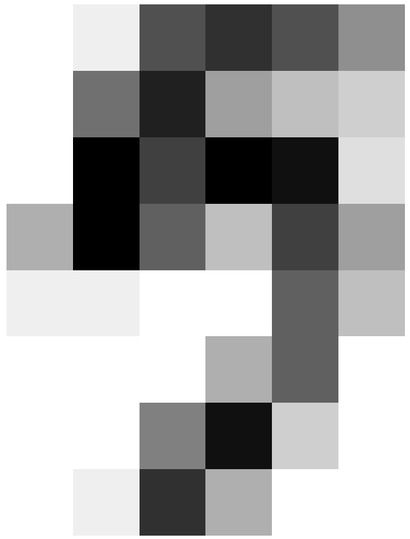


Captura do exercício: dados sintéticos, modelos treinados.

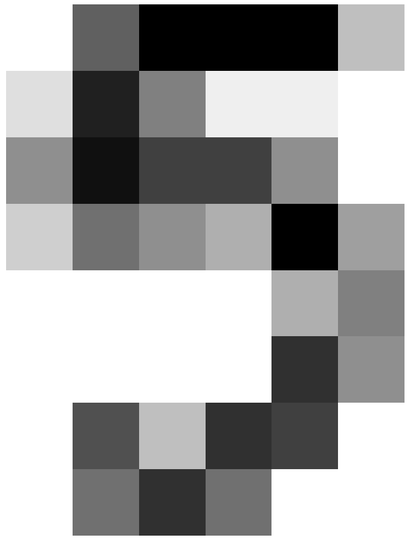
O problema: reconhecer dígitos

Imagens diferentes devem levar à mesma resposta: o dígito 5.

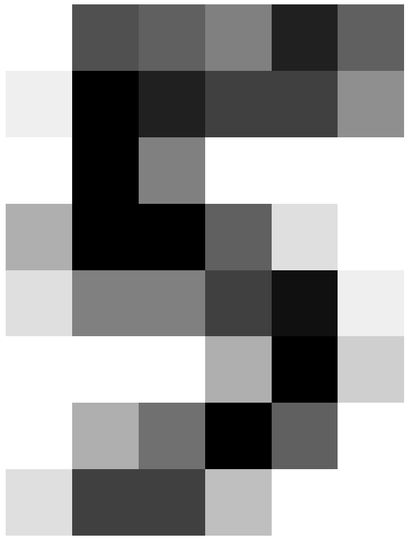
Escrita A



Escrita B



Escrita C

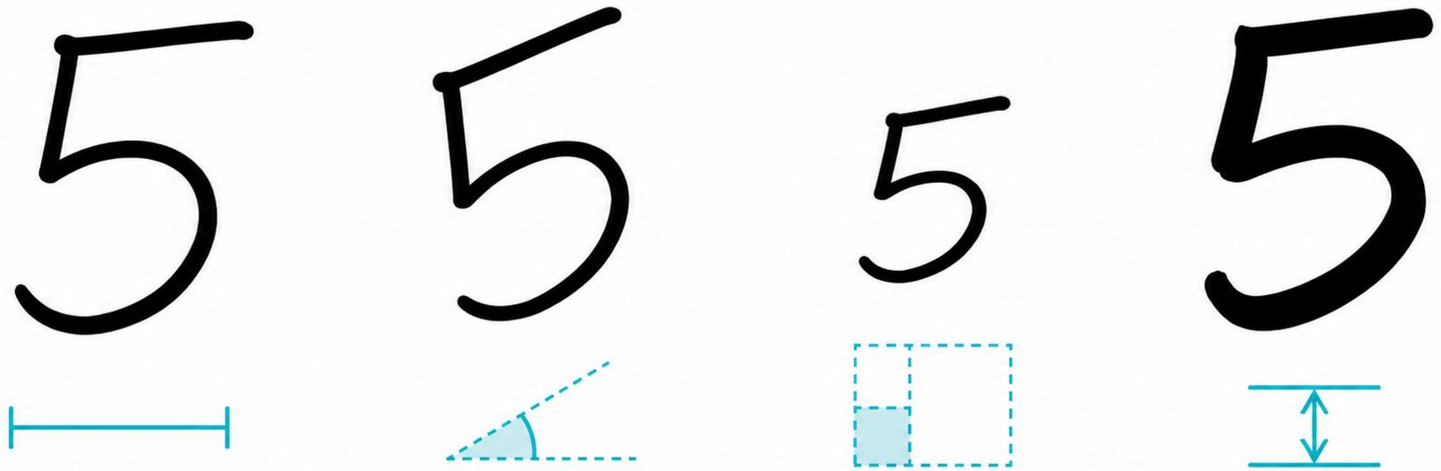


Como reconhecer uma nova forma de escrever o mesmo dígito?

Dados reais: UCI Optical Digits · conjunto de treino · CC BY 4.0

Variações da escrita

Regras sobre forma e posição têm de lidar com inclinação, tamanho e espessura.



O que muda?

Inclinação, posição, tamanho e espessura do traço.

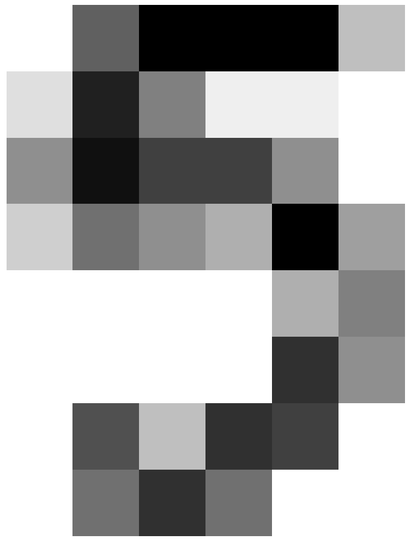
Descritores robustos

Devem aceitar diferentes escritas e manter diferenças entre classes.

A imagem como matriz de intensidades

O mesmo exemplo pode ser visto como imagem ou como 64 valores numéricos.

Imagem 8 × 8



Matriz do mesmo exemplo

0	0	10	16	16	16	4	0
0	2	14	8	1	1	0	0
0	7	15	12	12	7	0	0
0	3	9	7	5	16	6	0
0	0	0	0	0	5	8	0
0	0	0	0	0	13	7	0
0	0	11	4	13	12	0	0
0	0	9	13	9	0	0	0

0: claro
16: escuro

8 / 16 = 0,5

8 × 8 = 64 entradas. Normalização: dividir cada valor por 16.

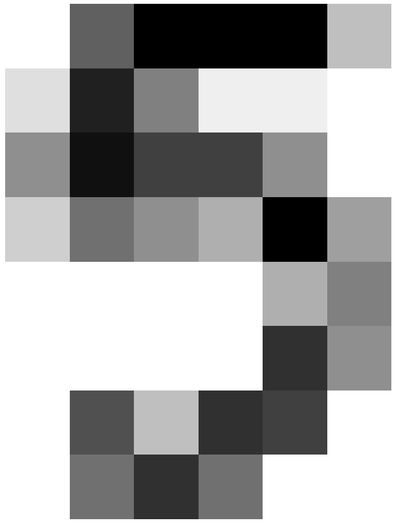
UCI Optical Digits · treino · mesma representação dos exercícios · CC BY 4.0

piep
CONVERTING IDEAS INTO SUSTAINABLE PRODUCTS

A matriz do dígito como vetor de entrada

Os 64 valores mantêm uma ordem fixa. Cada um passa a ser uma entrada x_i.

Imagem 8 × 8



Ler linha a linha

Primeira linha: [0, 0, 10, 16, 16, 16, 4, 0]

Dividir por 16

x = [0; 0; 0,625; 1; 1; 1; 0,25; 0;
0; 0,125; 0,875; 0,5; …; 0]

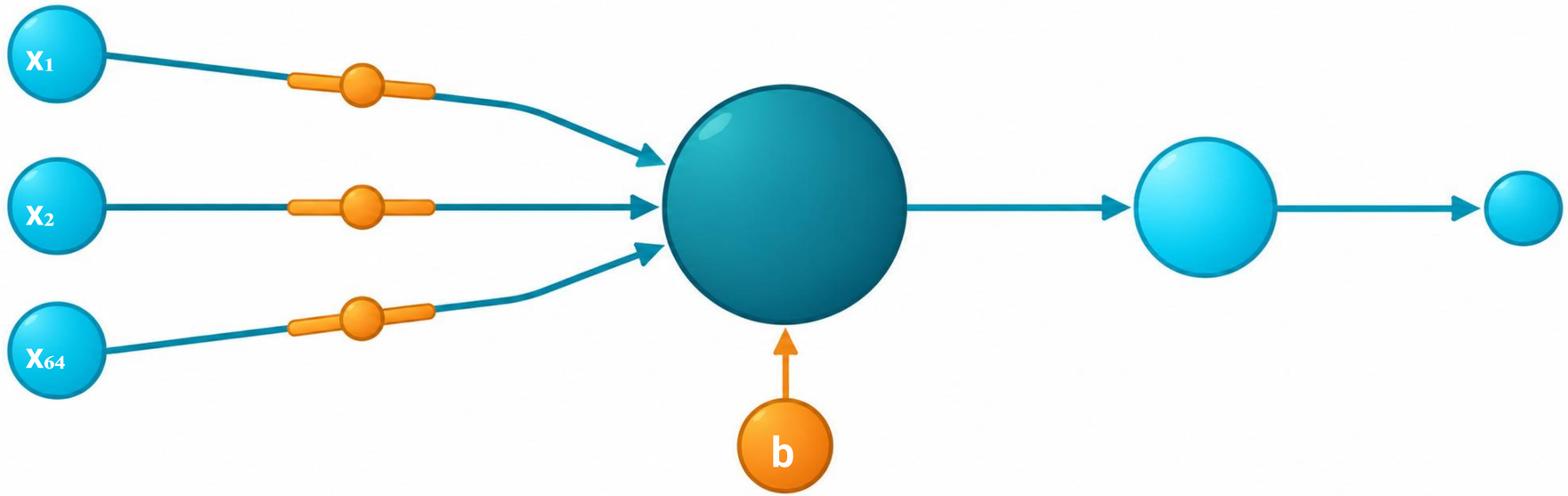
x_1 … x_8 são a primeira linha. x_9 começa a segunda.

Achatar muda a forma dos dados e conserva os 64 valores e a sua ordem.

O cálculo de um neurónio

Cada ligação tem um peso. O termo independente b desloca a soma antes da ativação.

Entradas x_i **Pesos w_i** **$z = \Sigma w_i x_i + b$** **$a = f(z)$** **Saída a**

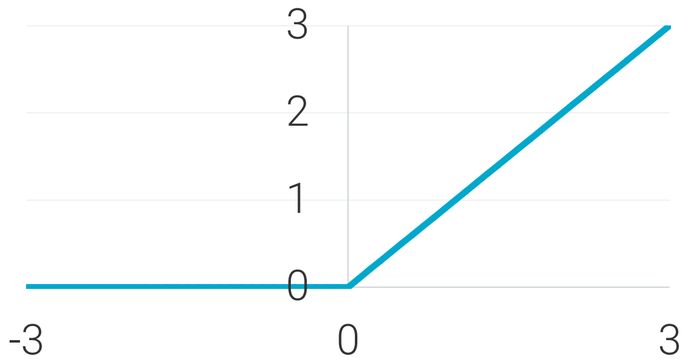


Funções de ativação

Cada função transforma a mesma entrada z de uma forma diferente.

ReLU

f(z) = max(0, z)

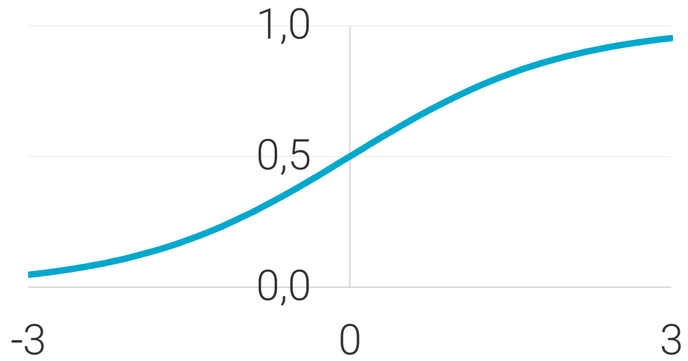


f(−1) = 0

f(1) = 1

Sigmoide

f(z) = 1 / (1 + exp(−z))

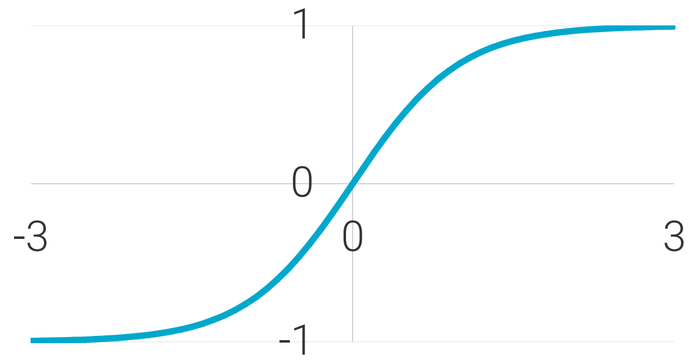


f(−1) = 0,269

f(1) = 0,731

Tanh

f(z) = tanh(z)

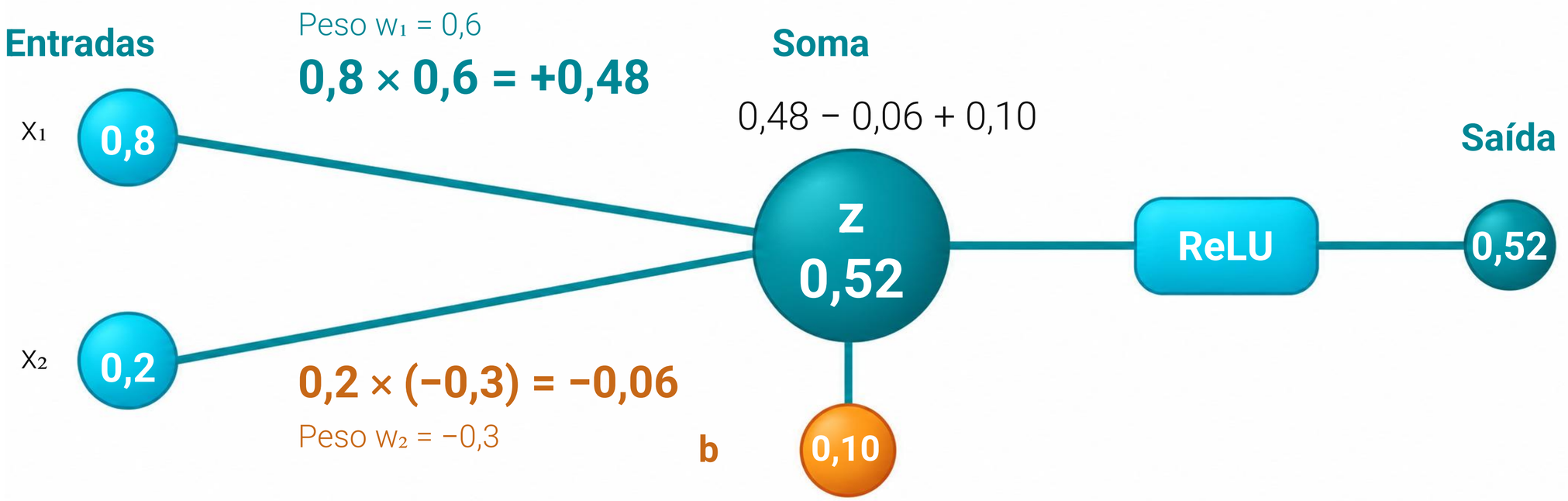


f(−1) = −0,762

f(1) = 0,762

O efeito dos pesos e do termo independente

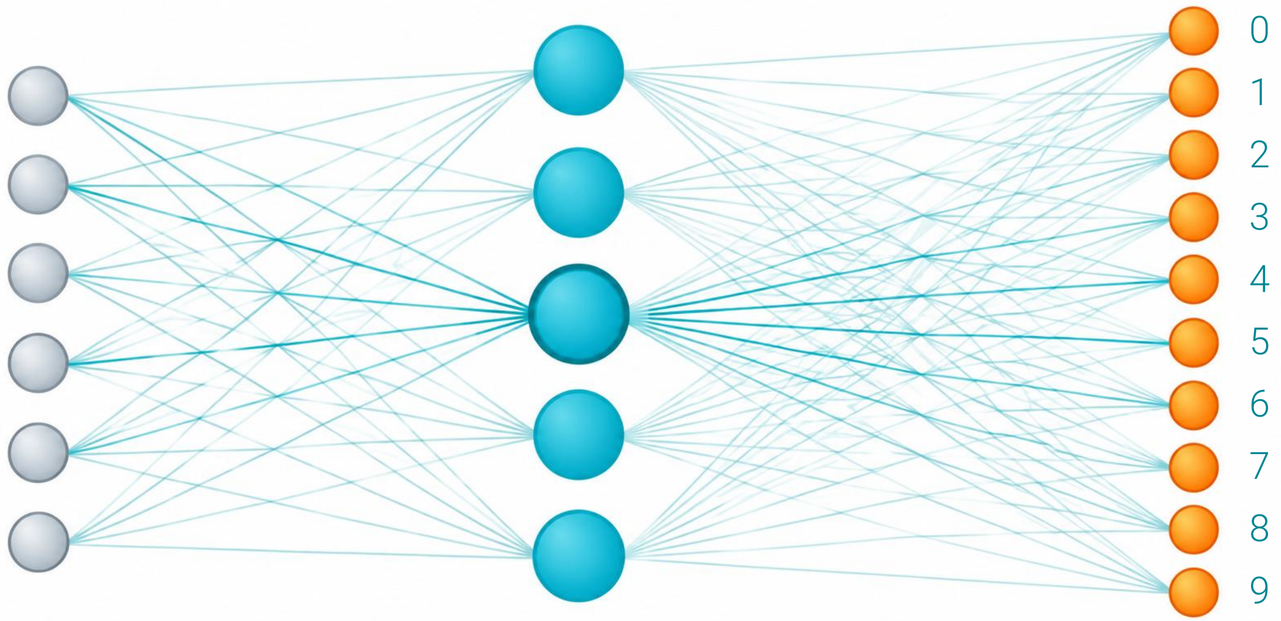
Cada entrada contribui com o seu valor multiplicado pelo peso.



Neurónios interligados em camadas

Cada neurónio recebe as saídas da camada anterior e tem os seus próprios parâmetros.

64 entradas x_i **32 neurónios · ReLU** **10 pontuações**



Uma pontuação por classe

Dígitos 0 a 9

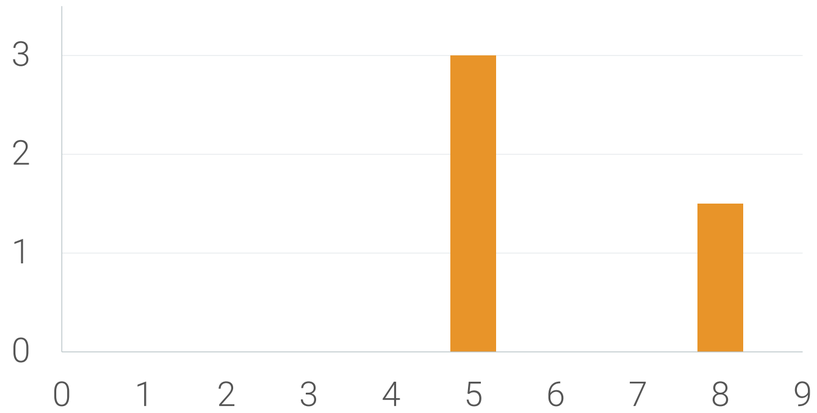
As saídas de uma camada são as entradas da seguinte.

Ligação densa: cada unidade recebe todas as entradas da camada anterior. Mostramos só parte das 64 e das 32 unidades.

Softmax e probabilidades das classes

A camada final produz 10 pontuações, um por dígito. A softmax considera todos em conjunto.

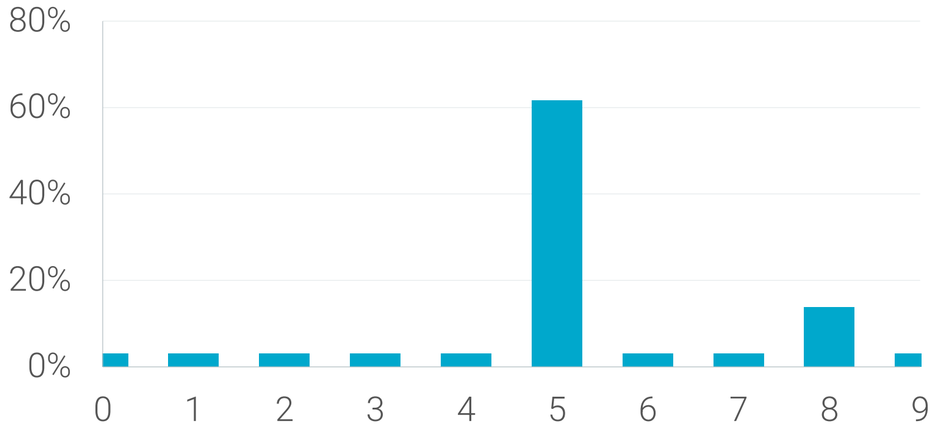
Pontuações ilustrativos



Softmax

Exponenciar
e dividir
pelo total

Probabilidade de cada classe



$p_k = \exp(s_k) / \Sigma_j \exp(s_j)$

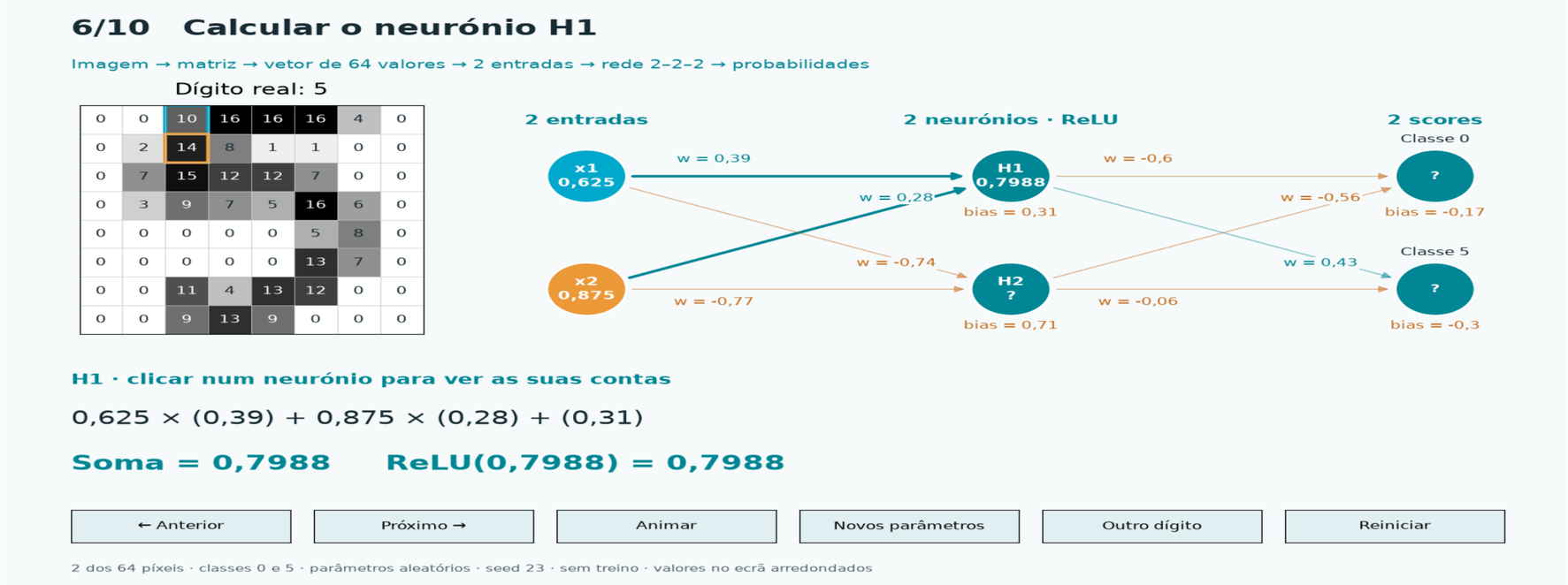
Para o 5: exp(3) / [8 + exp(3) + exp(1,5)] ≈ 61,7 %

Previsão: 5 · p_5 ≈ 61,7 %

Todas as probabilidades somam 1. Maior pontuação dá maior probabilidade. A previsão pode estar errada.

Passagem direta: uma rede para observar

Exercício 02: simplificamos a rede para conseguir seguir todas as contas.



2 entradas
2 neurónios ReLU
2 pontuações: 0 e 5

Parâmetros fixos durante a passagem

A rede completa 64–32–10 será treinada no exercício 05.

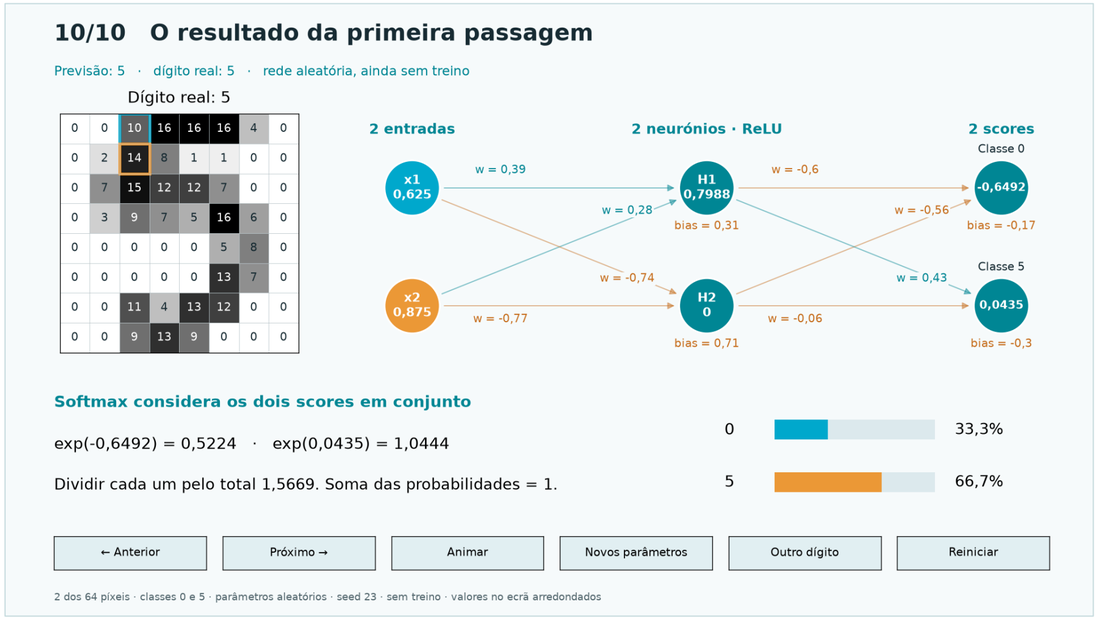
Demonstração: passagem direta

Demonstração · exercício 02

```
python 02_forward_pass_mlp.py
```

O que observar

- Dígito ⟶ matriz ⟶ vetor e duas entradas.
- Seguir os pesos e o cálculo de cada neurónio.
- A softmax dá probabilidades antes de haver treino.



Captura do exercício: primeira passagem, sem treino.

Perda: medir a qualidade da previsão

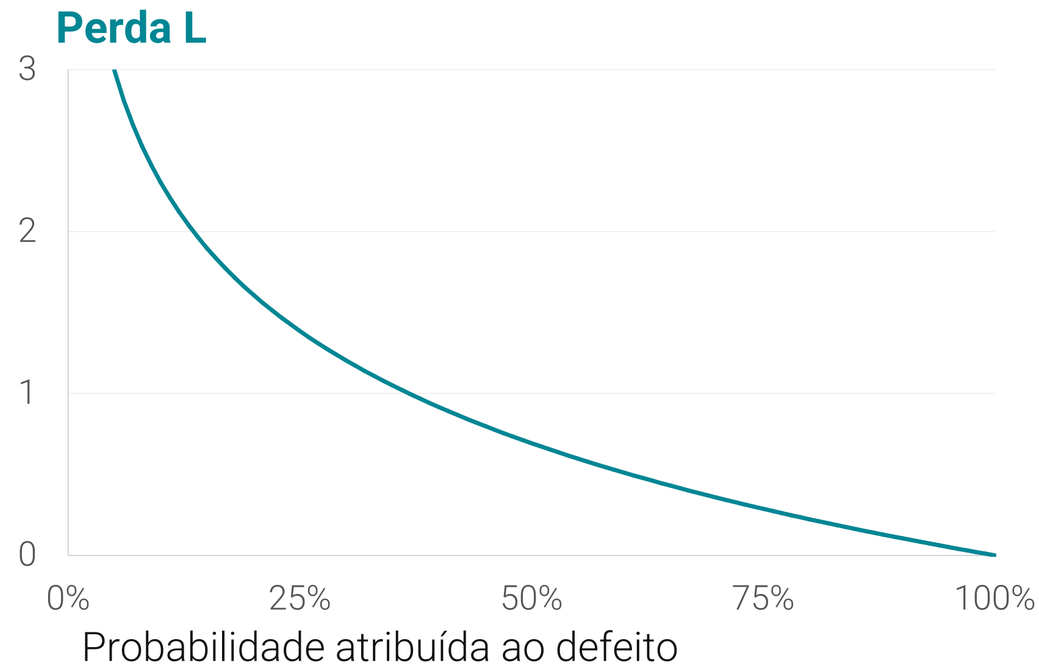
Exemplo binário «defeito»: y = 1 e p = 0,5 numa saída sigmoide.

A peça tem defeito

A rede atribui-lhe 50 % de probabilidade.

Para y = 1: $L = -\ln(p)$

$L = -\ln(0{,}5) \approx 0{,}693$

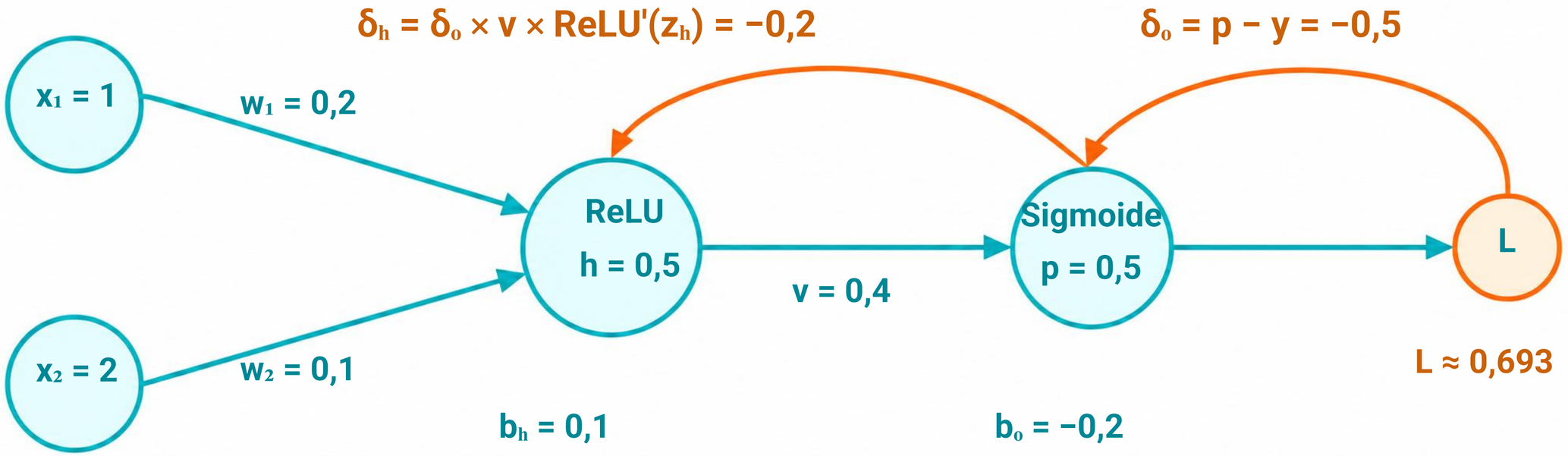


10 % → 2,303 50 % → 0,693 90 % → 0,105

Maior probabilidade da resposta correta → menor perda.

Retropropagação: da perda aos gradientes

Aplicamos a regra da cadeia a cada parâmetro, percorrendo o cálculo ao contrário.



Calculamos sensibilidades. Os parâmetros continuam iguais neste passo.

Um gradiente, passo a passo

Quanto varia a perda se alterarmos ligeiramente o peso w_1?

$$\partial L/\partial w_1 = (p - y) \times v \times \mathrm{ReLU}'(z_h) \times x_1$$

Gradiente da saída		**Peso da saída**		**Derivada da ReLU**		**Entrada**
−0,5	×	**0,4**	×	**1**	×	**1**

$\partial L/\partial w_1 = -0{,}2$

Aqui, aumentar um pouco w_1 tende a reduzir a perda.

Na saída: $\partial L/\partial v = -0{,}25$ $\partial L/\partial b_o = -0{,}5$

Na camada oculta: $\partial L/\partial w_2 = -0{,}4$ $\partial L/\partial b_h = -0{,}2$

Otimização: dar um passo de aprendizagem

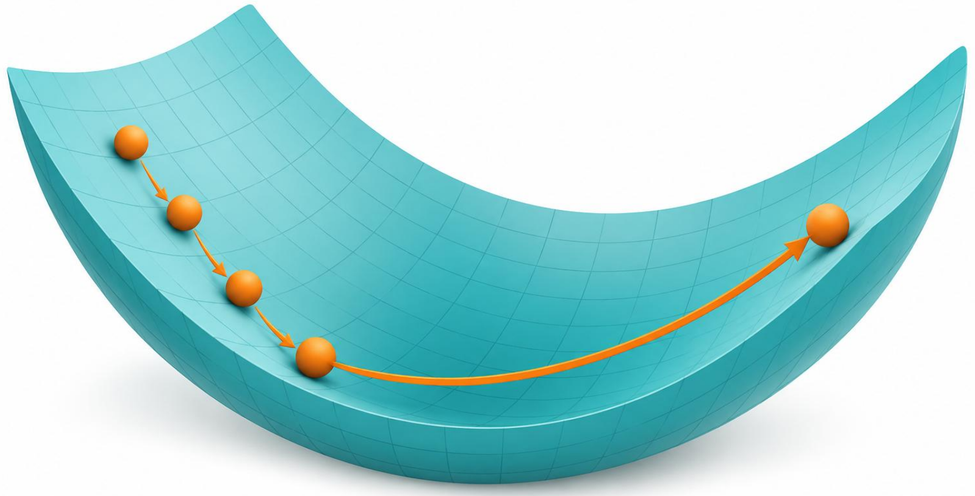
Usamos descida do gradiente: cada parâmetro dá um passo no sentido oposto ao gradiente.

Novo = atual − α × gradiente

Com taxa de aprendizagem α = 0,1:

w_1 novo = 0,2 − 0,1 × (−0,2)

w_1: 0,20 → 0,22

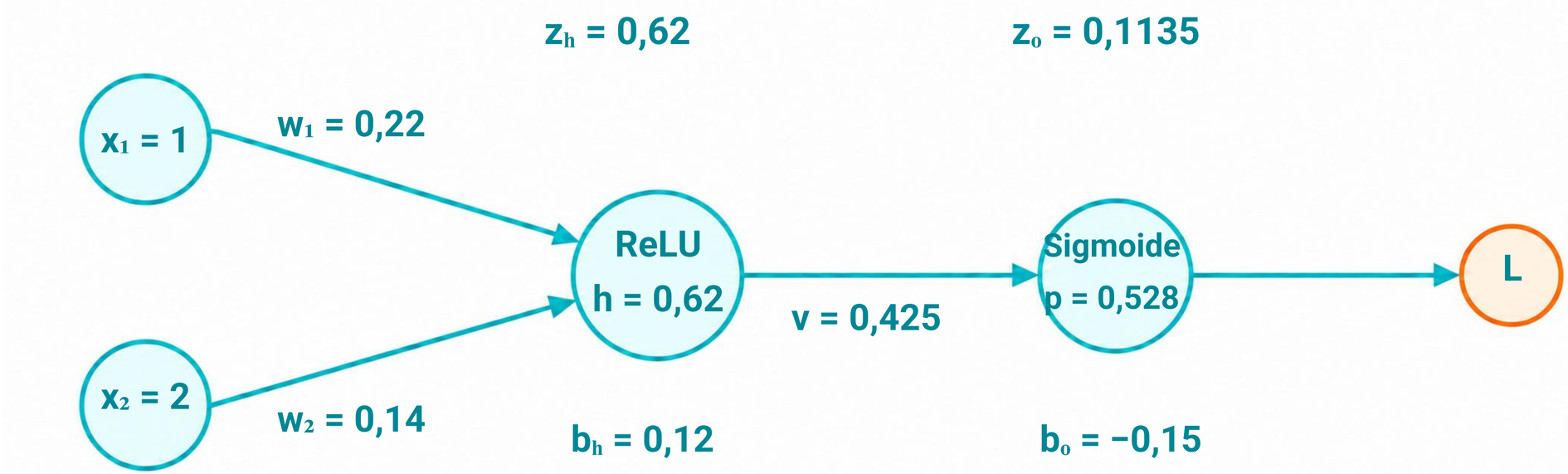


α controla o tamanho do passo.
Um passo excessivo pode aumentar a perda.

A mesma regra atualiza todos os parâmetros com os gradientes calculados.

Novos parâmetros, nova previsão

Após atualizar os cinco parâmetros com α = 0,1, repetimos a passagem direta com x = [1, 2].



Probabilidade: 0,500 → 0,528

Perda: 0,693 → 0,638

Exercício 03: dígitos 0/5, duas saídas softmax e perda −ln(p da classe correta).
Repetimos o mesmo ciclo: previsão, perda, gradientes e atualização.

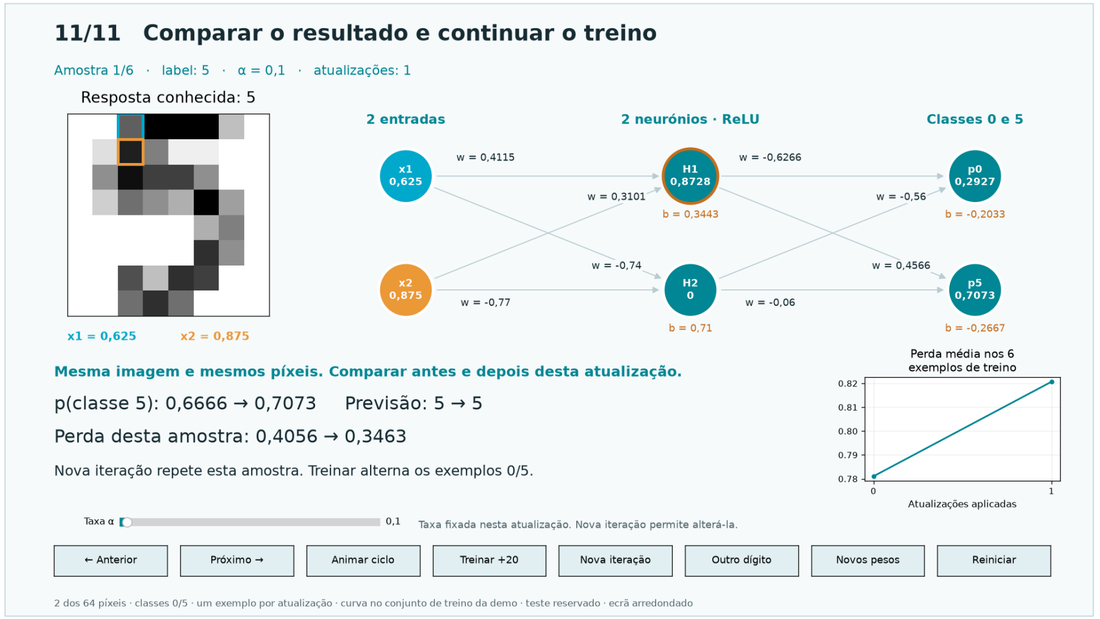
Demonstração: aprender, passo a passo

Demonstração · exercício 03

```
python 03_perda_backpropagation_otimizacao.py
```

O que observar

- Previsão, perda, gradientes e atualização.
- A taxa de aprendizagem controla o tamanho do passo.
- «Treinar +20» repete ciclos em vários exemplos.



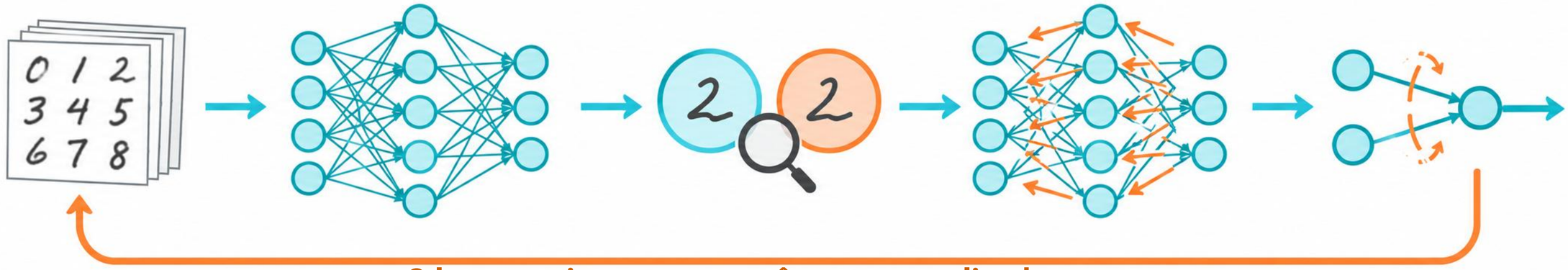
Captura do exercício: antes e depois de uma atualização.

Lotes, épocas e ciclo de treino

Um lote é um grupo de exemplos. Uma época percorre todos os lotes de treino.

Lote → **Passagem direta** → **Perda** → **Retropropagação** → **Atualização**

Imagens e rótulos



O lote seguinte usa os parâmetros atualizados.

1 000 exemplos ÷ 100 por lote = 10 atualizações por época

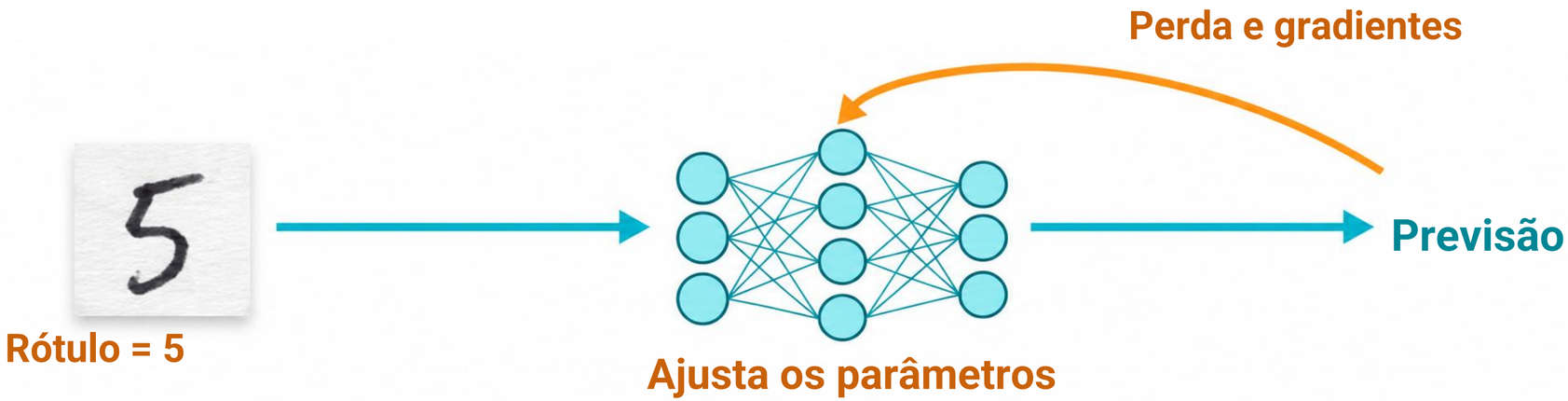
Na época seguinte, voltamos a percorrer os dados de treino.

Treino e inferência

A passagem direta calcula previsões em ambos. No treino supervisionado, usamos também os rótulos.

Treino

Imagem + resposta conhecida



Inferência

Nova imagem



Na inferência habitual, a nova imagem passa pela rede com os parâmetros aprendidos.

Treino, validação e teste

Separamos os exemplos antes de aprender os parâmetros ou escolher as opções de treino.



Treino

Aprender os parâmetros.

Validação

Escolher opções de treino e o momento de parar.

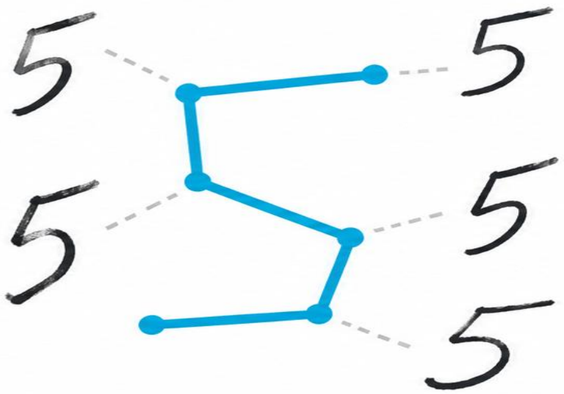
Teste

Estimar desempenho no final do desenvolvimento.

O teste fica reservado enquanto tomamos as decisões de desenvolvimento.

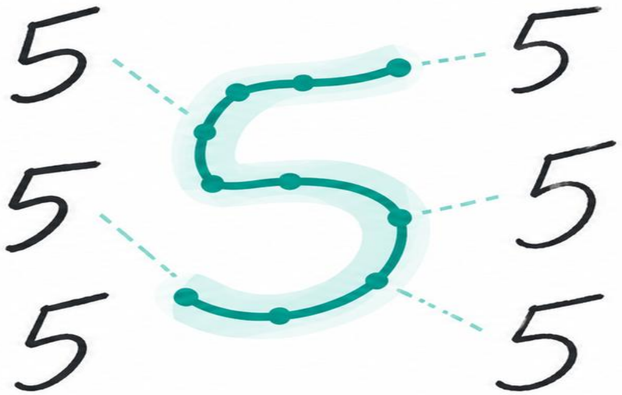
Generalização e capacidade do modelo

Generalizar é reconhecer exemplos novos, além dos usados para aprender os parâmetros.



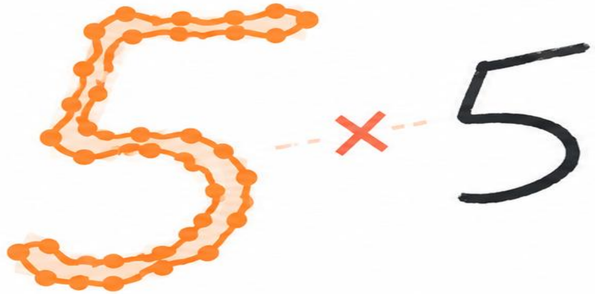
Subajustamento

Treino 60 %
Validação 58 %



Ajuste adequado

Treino 92 %
Validação 90 %



Sobreajustamento

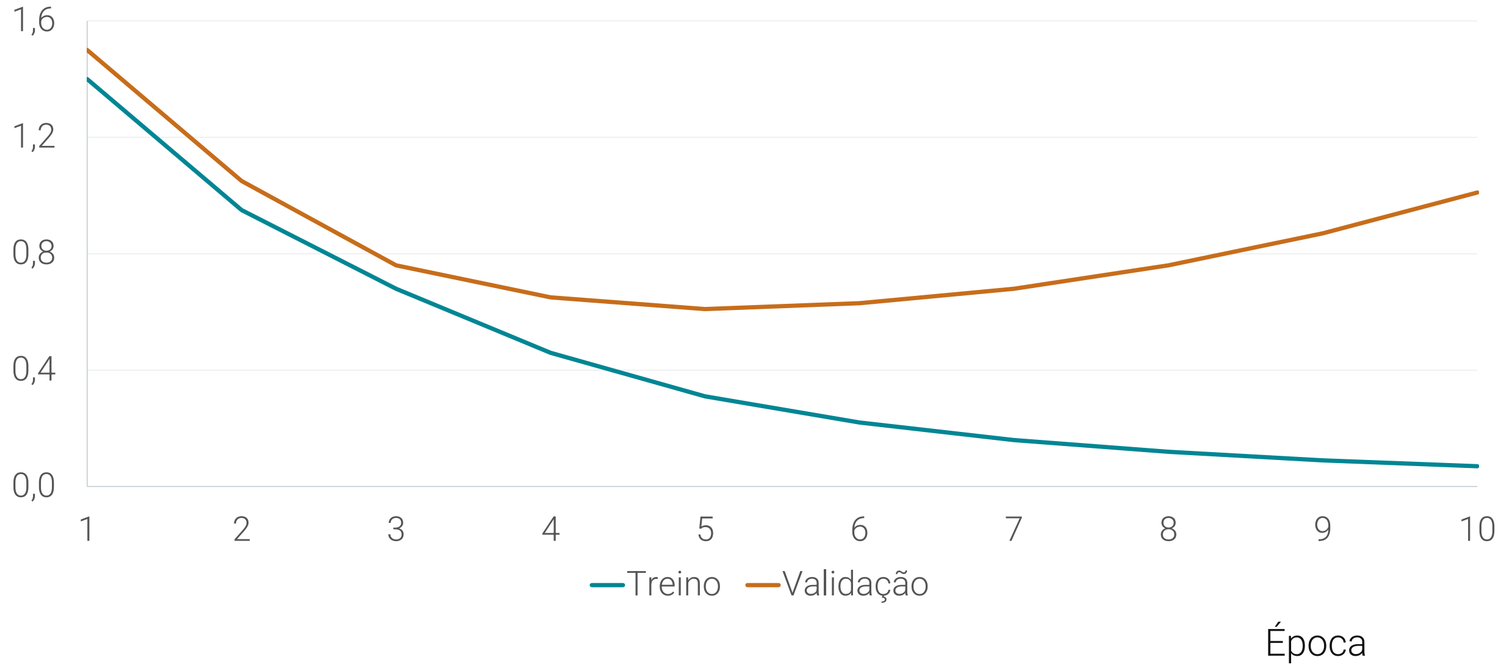
Treino 99 %
Validação 76 %

Valores fictícios para ilustrar padrões.

Curvas de treino e validação

A validação ajuda a escolher o momento de parar. O teste permanece reservado.

Perda média



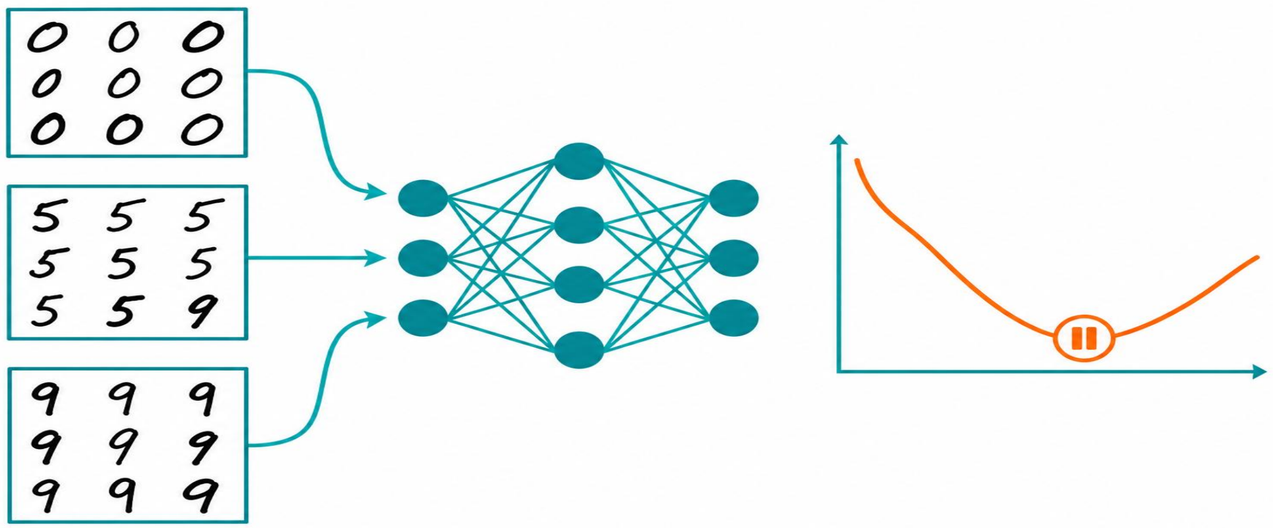
Época 5 menor perda de validação

Conservar os pesos desse momento. Continuar pode piorar a generalização.

Curvas fictícias para ilustrar o diagnóstico. A divergência não é inevitável.

Como reduzir o sobreajustamento

O diagnóstico na validação orienta as alterações ao treino.



Mais exemplos relevantes

Diversidade, qualidade e rótulos corretos.

Capacidade e regularização

Ajustar a rede. Penalizar pesos ou desativar neurónios.

Momento de parar

Guardar o modelo escolhido pela validação.

O exercício 05 usa uma rede pequena e guarda o modelo escolhido pela validação.

Matriz de confusão

100 peças fictícias. A classe positiva é «defeito». Cada previsão cai numa das quatro situações.

	Previsto: defeito	**Previsto: conforme**
Real: defeito	16 TP: detetado	4 FN: escapou
Real: conforme	8 FP: falso alarme	72 TN: correto

4 defeitos passaram despercebidos

8 peças conformes foram sinalizadas

Na Interface, clicar num grupo mostra quais as peças que contribuíram para essa contagem.

Taxa de acerto e classes raras

Taxa de acerto: de todas as previsões, quantas estavam corretas?

(TP + TN) / total

(16 + 72) / 100 = 88 %

Prever sempre «conforme»

98 % de taxa de acerto
0 % dos defeitos detetados



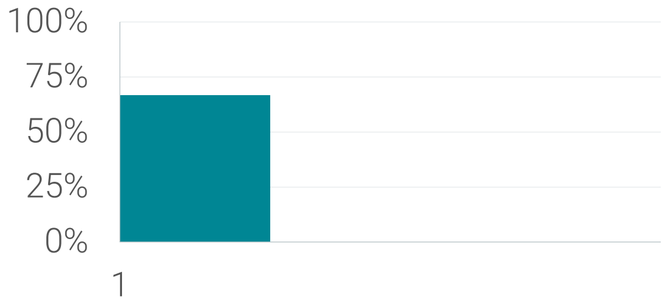
Outro conjunto: 98 conformes e 2 defeitos

Precisão e sensibilidade

As mesmas 16 deteções corretas respondem a perguntas diferentes.

Precisão

Dos sinalizados, quantos tinham defeito?

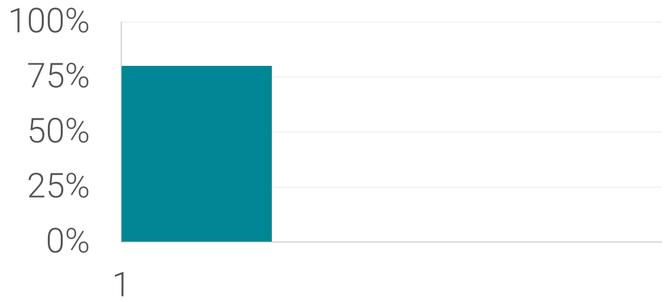


16 / (16 + 8)

66,7 %

Sensibilidade

Dos defeitos reais, quantos foram detetados?



16 / (16 + 4)

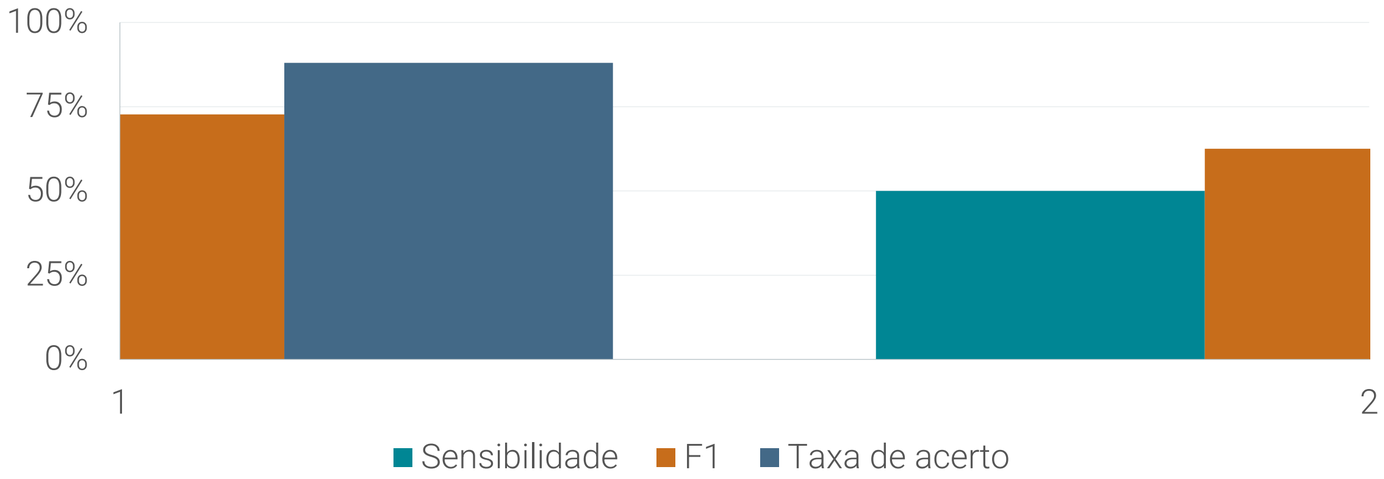
80 %

Precisão conta falsos alarmes no denominador. Sensibilidade conta os defeitos que escaparam.

F1 e comparação de modelos

F1 combina precisão e sensibilidade. Dois modelos podem ter a mesma taxa de acerto e erros diferentes.

F1 = 2 × precisão × sensibilidade / (precisão + sensibilidade)



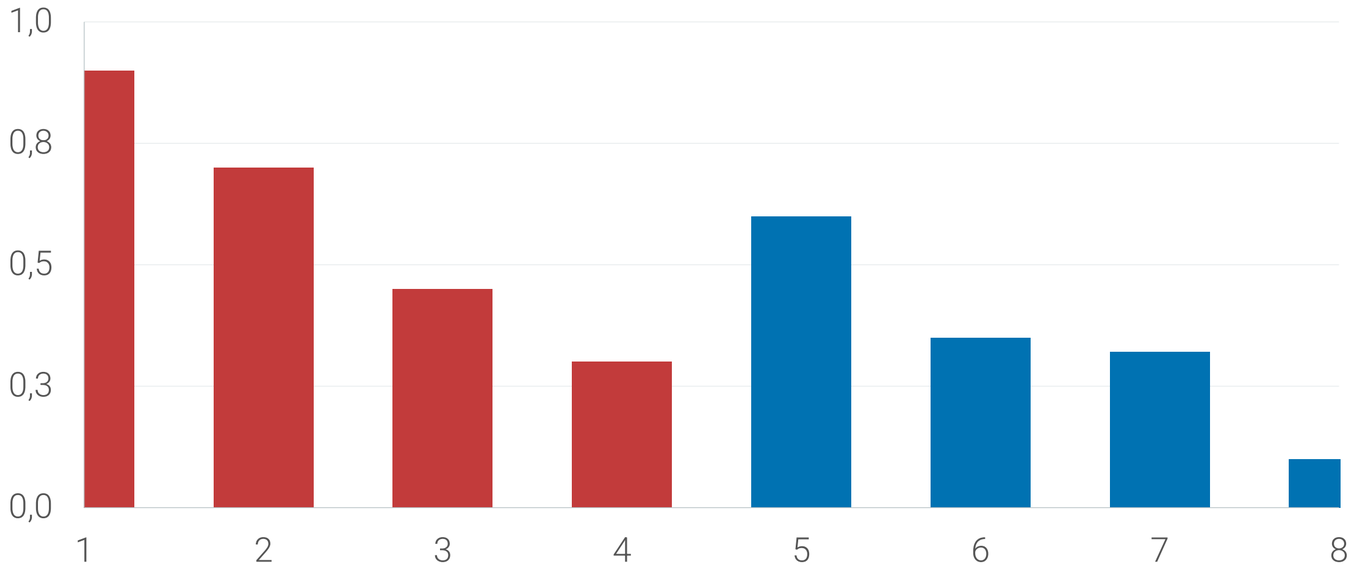
1: modelo A 2: modelo B

Modelo A
F1 ≈ 72,7 %
4 defeitos escapam

Modelo B
F1 = 62,5 %
10 defeitos escapam

O limiar muda as decisões

As pontuações ficam iguais. Mudamos apenas o limiar para sinalizar uma peça.



Peças 1–4: defeito. Peças 5–8: conforme.

Limiar 0,70
Precisão 100 %
Sensibilidade 50 %

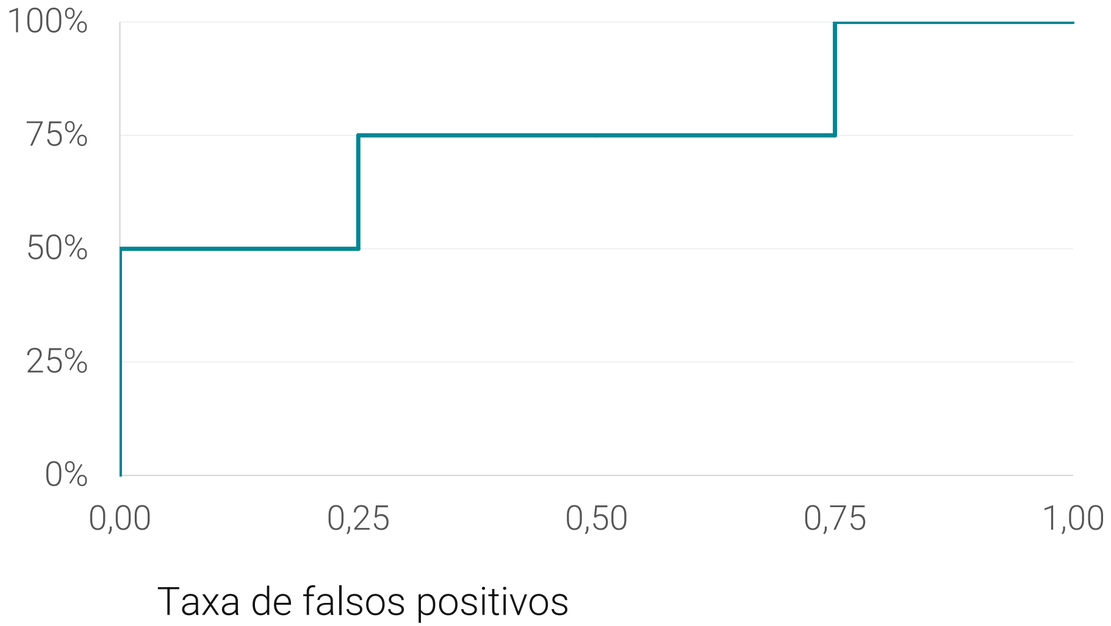
Limiar 0,30
Precisão 57,1 %
Sensibilidade 100 %

Sinalizar quando pontuação ≥ limiar. Na Interface, arrastar o limiar e observar as peças que mudam.

Limiar, ROC e custo dos erros

A curva ROC reúne decisões obtidas com vários limiares. O custo dos erros orienta a escolha.

Sensibilidade



Custos fictícios dos modelos A e B

Defeito que escapa custa 10. Alarme custa 1.
A: 48 unidades B: 102 unidades

Defeito que escapa custa 1. Alarme custa 10.
A: 84 unidades B: 30 unidades

AUC resume a ordenação das pontuações. Não escolhe, por si só, o limiar ou o custo aceitável.

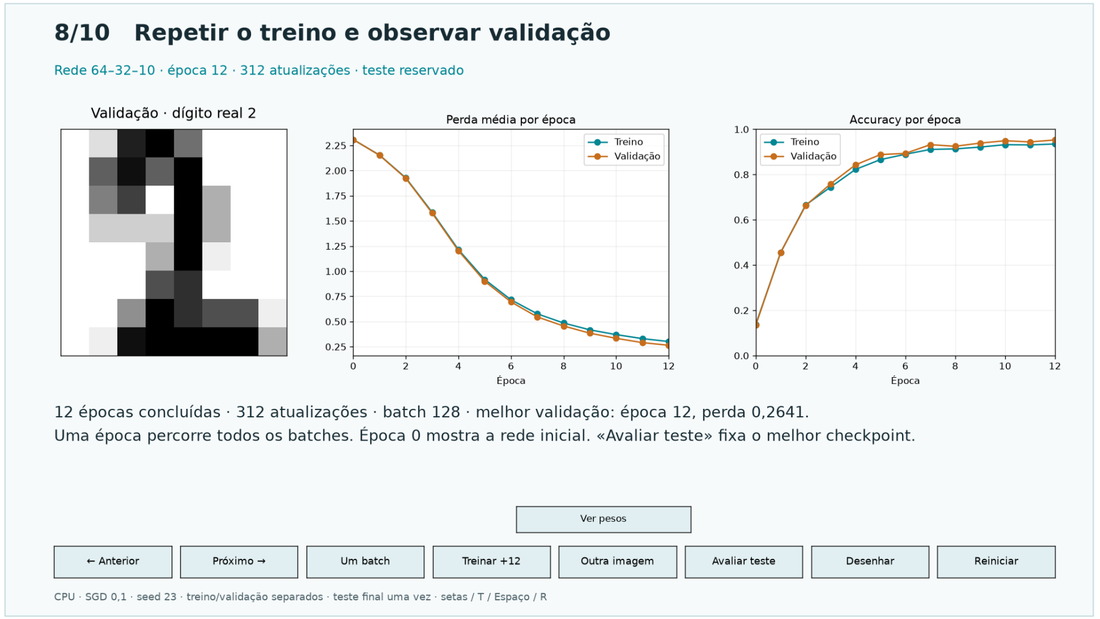
Demonstração: classificador de dígitos

Demonstração · exercício 05

```
python 05_mlp_digitos.py
```

O que observar

- Treinar a rede 64–32–10 e acompanhar as curvas.
- Escolher o modelo pela validação e avaliar o teste.
- Desenhar um dígito e observar a previsão.



Captura do exercício: curvas após 12 épocas.

Visão computacional: classificação e deteção

Uma categoria para a imagem inteira, ou uma classe e uma localização para cada objeto.

Classificação



Classe prevista: Samoiedo (Samoyed)
Confiança: 36,3%

Deteção



Classe prevista: cão (dog)
A caixa localiza o objeto.

Segmentação de objetos e pose humana

A segmentação marca regiões de cada objeto. A pose estima pontos do corpo das pessoas.

Segmentação de instâncias



Máscaras
por objeto

Píxeis do
autocarro e
das pessoas

Pose humana



Pontos do corpo
e ligações

Cabeça, ombros,
cotovelos,
anca e pernas

Caixas orientadas e demonstração com YOLO

Um modelo por tarefa. Na demonstração, escolhemos também imagem, vídeo ou câmara.



Caixas orientadas

As caixas acompanham
a orientação dos barcos.
Modelo treinado
em imagens aéreas.

Exercício 06

Escolher tarefa e fonte.
Analisar e comparar.
No vídeo, pausar ou avançar um fotograma.

Inferência usa pesos já aprendidos. No módulo seguinte, retomamos a ideia com texto.

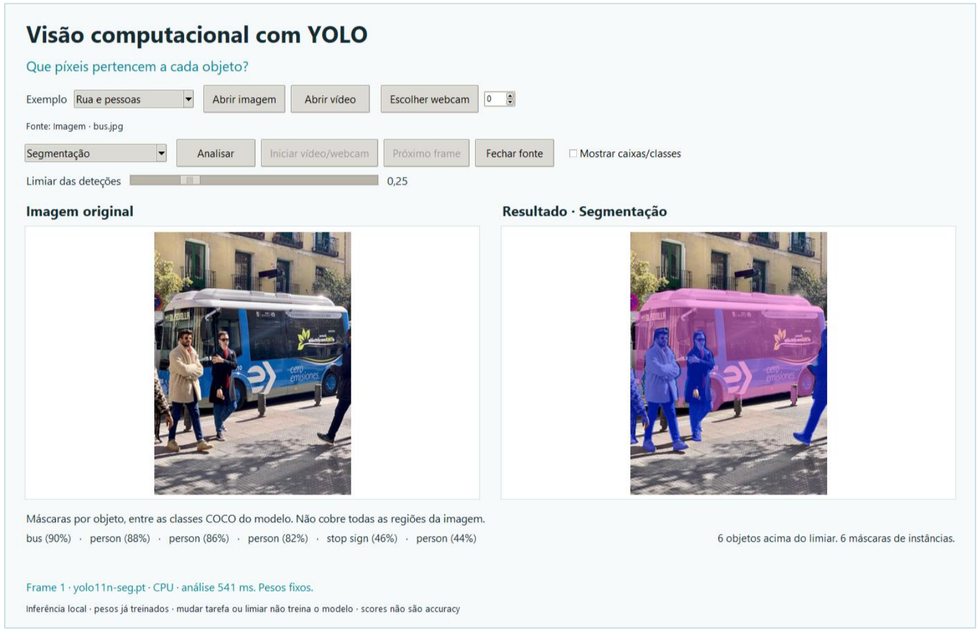
Demonstração: visão computacional com YOLO

Demonstração · exercício 06

```
python 06_visao_computacional_yolo.py
```

O que observar

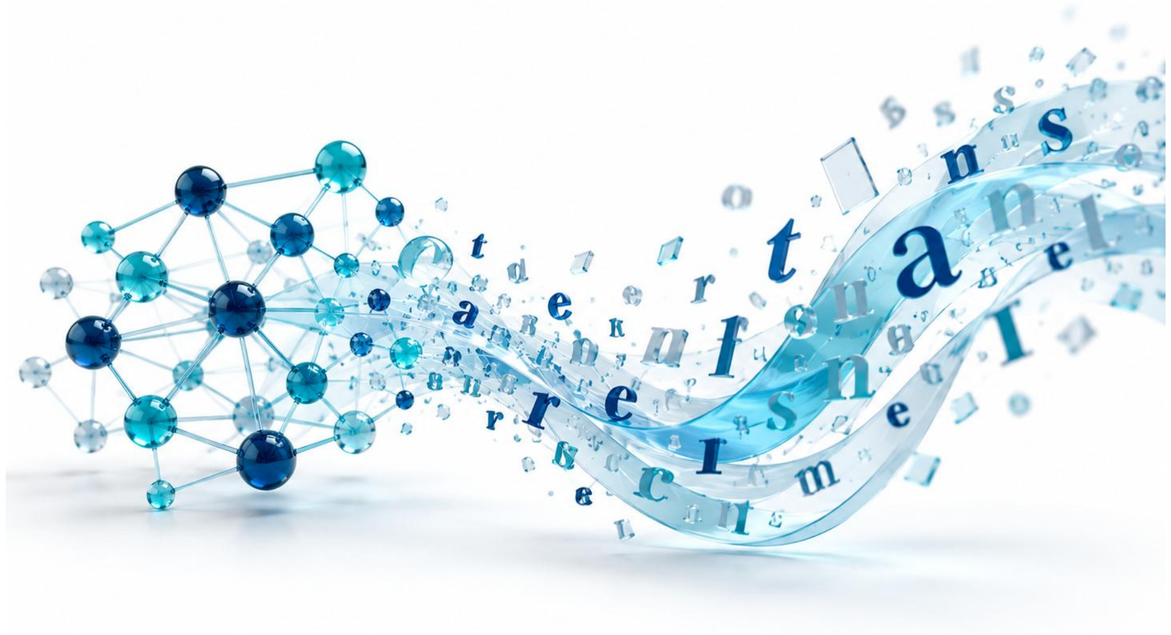
- Escolher classificação, deteção, segmentação ou pose.
- Comparar também caixas orientadas e o limiar.
- Experimentar imagem, vídeo e câmara.



Captura do exercício: segmentação numa imagem local.

Módulo 2

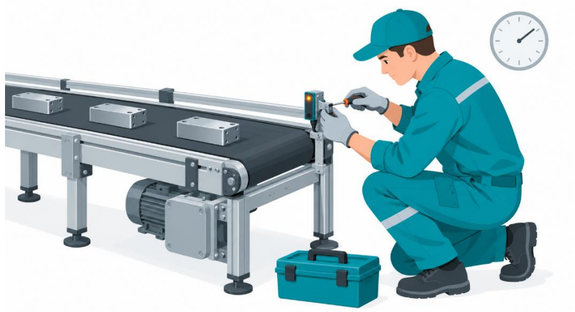
De redes neuronais a modelos de linguagem



Objetivos do módulo 2

Como funciona um modelo de conversação com decoder causal?

1. Texto em números	2. Arquitetura	3. Gerar	4. Treinar e adaptar	5. Usar e avaliar	6. Multimodal
tokens e embeddings	atenção e Transformer	prever e repetir	aprender e adaptar	contexto e controlos	imagem e texto



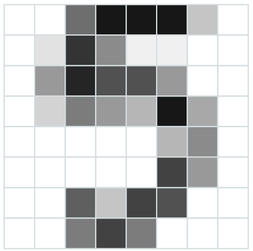
Fio condutor

«A linha 2 parou às 10h15 devido a uma falha no sensor. A produção foi retomada às 10h35 após a substituição do sensor.»

O que trazemos do módulo 1

Módulo 1 · classificar um dígito

64 números



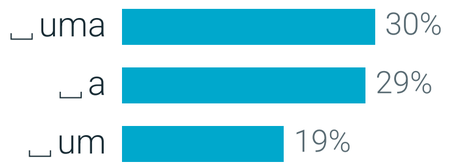
→ Rede neuronal →

Probabilidade de cada dígito (0–9)

Módulo 2 · continuar um texto

«A produção foi retomada após»

→ Rede neuronal →



Em ambos: números ⟶ rede com pesos ⟶ probabilidades.

Modelos de linguagem e tarefas

Um decoder causal processa o pedido e gera a resposta. Pode classificar, resumir ou continuar texto.

«A linha 2 parou às 10h15 devido a uma falha no sensor. A produção foi retomada às 10h35 após a substituição do sensor.»

Classificar por geração
Manutenção

Resumir por geração
A linha 2 esteve parada 20 minutos devido a uma falha no sensor.

Vamos acompanhar um Transformer só com decoder. Exemplo fictício.

Tokens e IDs: do texto aos números

REPRESENTAR: encoders e decoders começam por transformar tokens em vetores.

O tokenizador parte o texto em peças; cada peça tem um número: o ID.

Texto A peça parou e a peça partiu.

Tokens	A	pe	##ça	par	##ou	e	a	pe	##ça	par	##tiu	.
IDs	37	**905**	**2210**	518	77	9	12	**905**	**2210**	518	3120	4

A mesma peça tem sempre o mesmo ID. O ID é um número de catálogo: não é a posição nem a importância.

Vocabulário didático: ## marca a continuação de uma palavra.

Demonstração: tokens e IDs

REPRESENTAR: encoders e decoders começam por transformar tokens em vetores.

Demonstração · exercício 01

```
python 01_tokens_ids.py
```

O que observar

- Uma palavra nova parte-se em várias peças ou fica [UNK].
- A mesma peça tem sempre o mesmo ID.
- «Contar no AMALIA»: o modelo real usa outro vocabulário.



Captura do exercício (respostas gravadas dos modelos).

Embeddings: dos IDs aos vetores

REPRESENTAR: encoders e decoders começam por transformar tokens em vetores.

##ça

↓

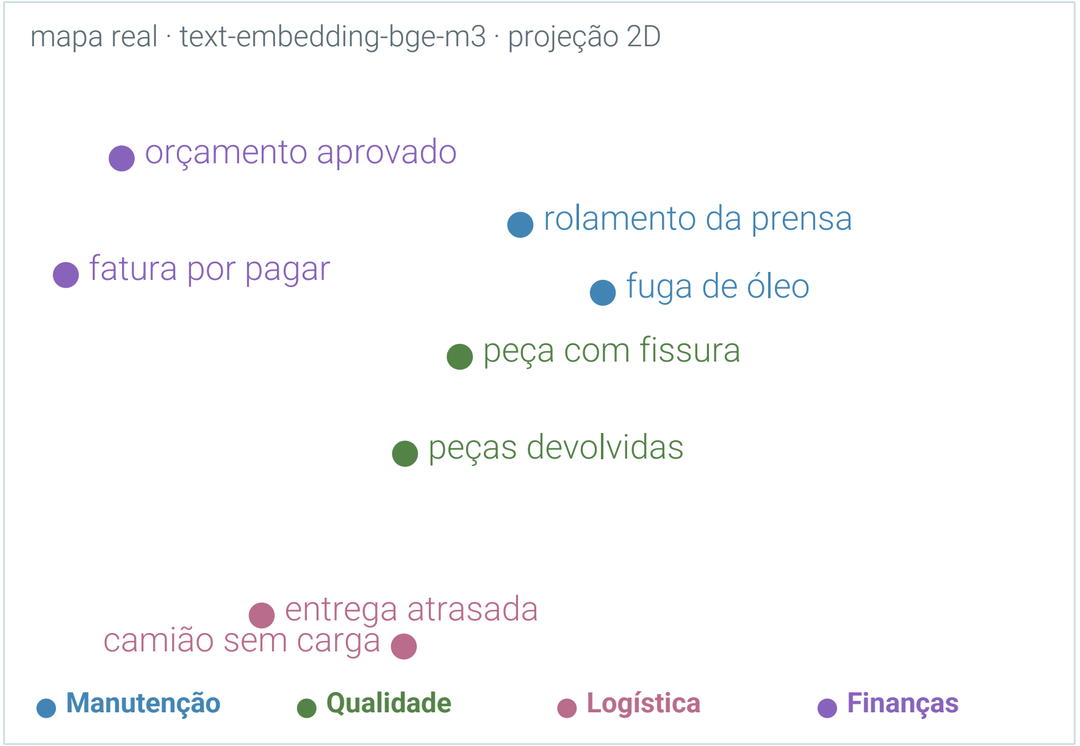
ID 2210

↓

2209	0,12	−0,40	0,77	…
2210	**0,41**	**−0,27**	**0,08**	**…**
2211	−0,63	0,15	0,30	…

O ID seleciona um vetor na tabela de embeddings. Encoders e decoders usam esta representação.

À direita: embeddings de frases produzidos pelo bge-m3, projetados em 2D.



Significado, contexto e posição

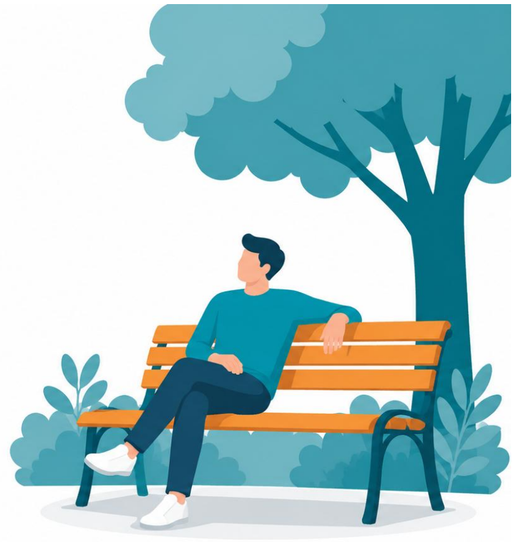
REPRESENTAR: encoders e decoders começam por transformar tokens em vetores.

A mesma palavra, dois sentidos

Fui ao **banco** levantar dinheiro.

Sentei-me no **banco** do jardim.





As mesmas palavras, outra ordem

O	cão	perseguiu	o	gato
1	2	3	4	5

O	gato	perseguiu	o	cão
1	2	3	4	5

O modelo precisa da posição para distinguir estas duas frases.

Demonstração: mapa de vetores

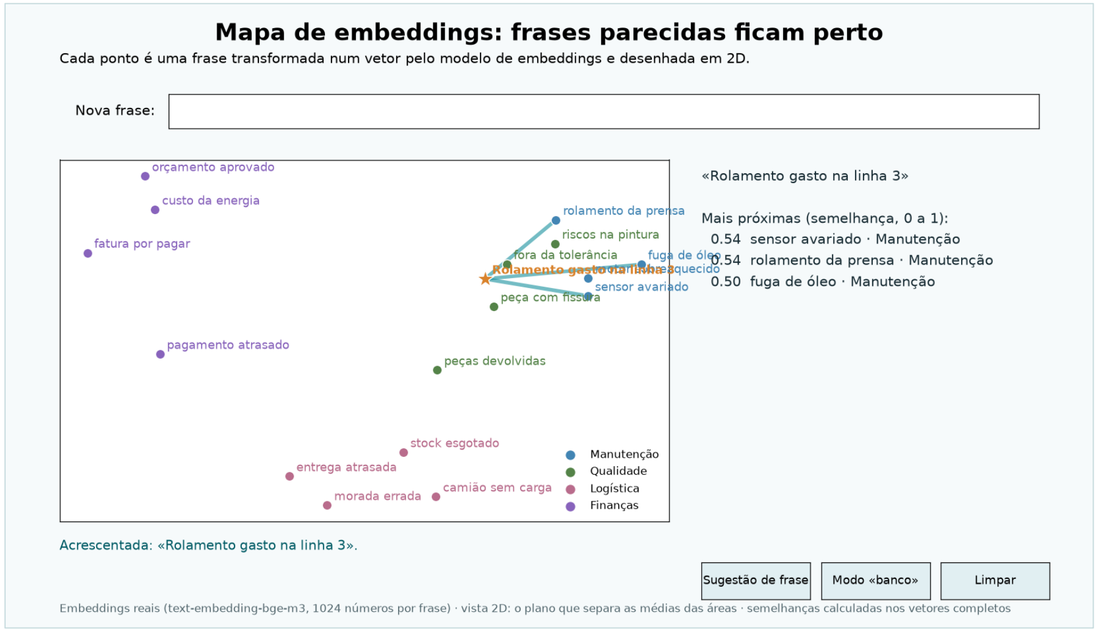
REPRESENTAR: encoders e decoders começam por transformar tokens em vetores.

Demonstração · exercício 02

```
python 02_mapa_embeddings.py
```

O que observar

- Textos da mesma área aparecem próximos.
- Acrescentar um termo novo e ver onde fica.
- Modo «banco»: a frase muda de lugar com o contexto.

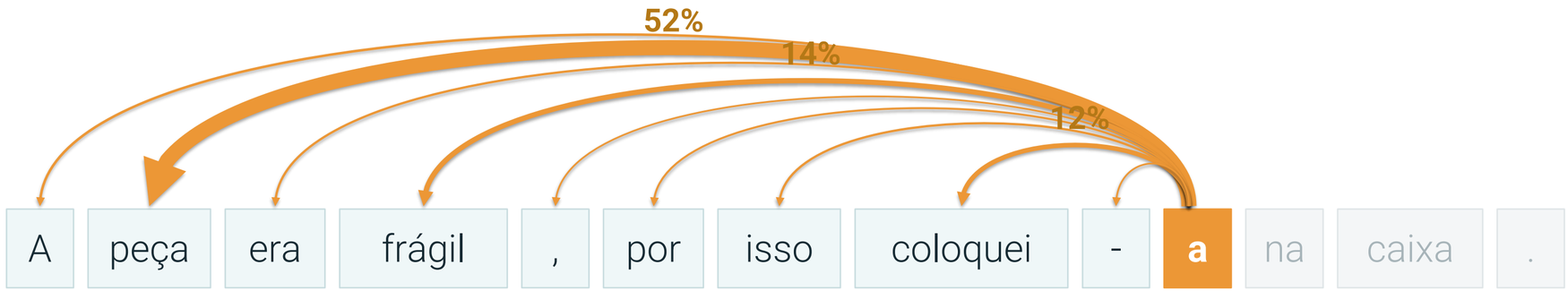


Captura do exercício (respostas gravadas dos modelos).

Atenção: combinar informação do contexto

PROCESSAR: a atenção combina contexto e a arquitetura define as consultas permitidas.

Cada token combina informação de outros tokens com pesos de atenção.



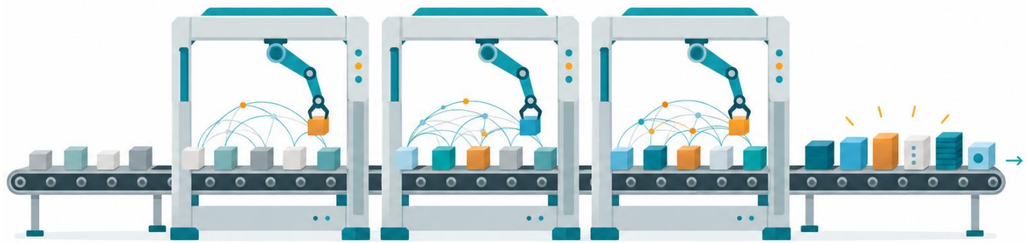
Consulta causal até «a».

A representação de «a» combina vetores de valor:
52% de «peça» + 14% de «frágil» + 12% de «coloquei» + …

Pesos didáticos. As consultas permitidas dependem da arquitetura.

Transformer: organizar atenção e processamento

PROCESSAR: a atenção combina contexto e a arquitetura define as consultas permitidas.



Embeddings dos tokens + informação de posição

↓

Bloco Transformer · repete-se em várias camadas

Atenção: combina informação entre tokens

Rede de propagação direta: processa cada posição

↓

Representações contextualizadas

Encoder, decoder e encoder–decoder

PROCESSAR: a atenção combina contexto e a arquitetura define as consultas permitidas.

Encoder



Cada token consulta toda a entrada.

Usos típicos:
representar, classificar, pesquisar

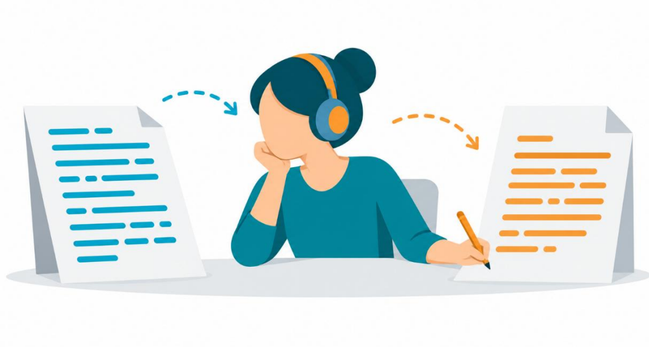
Decoder causal



Cada token consulta apenas até à sua posição.

Usos típicos:
gerar, resumir, classificar

Encoder + decoder



O decoder consulta a saída do encoder.

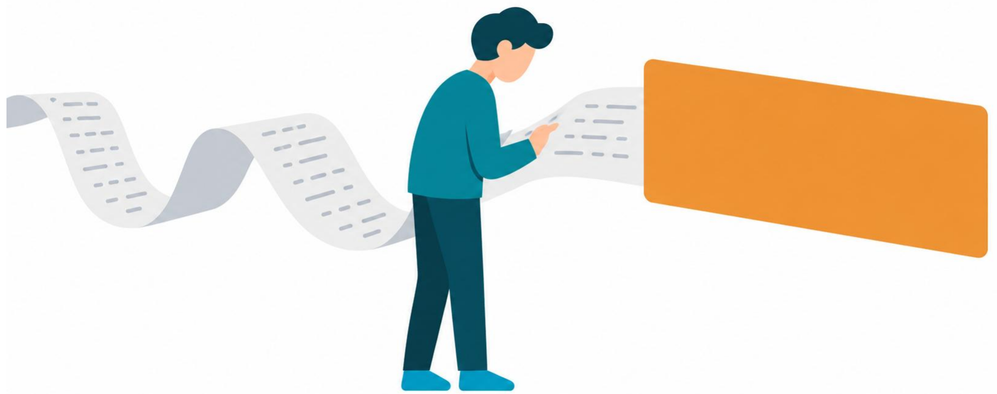
Usos típicos:
traduzir e gerar a partir da entrada

Atenção causal: até ao token atual

PROCESSAR: a atenção combina contexto e a arquitetura define as consultas permitidas.

quem consulta ↓ \ token consultado →	A	peça	era	frágil
A	100%	—	—	—
peça	20%	80%	—	—
era	10%	60%	30%	—
frágil	5%	65%	10%	20%

Cada linha soma 100%. Mais escuro = maior peso. — futuro bloqueado.



Pesos didáticos de uma cabeça de atenção. O futuro recebe peso zero.

Demonstração: atenção

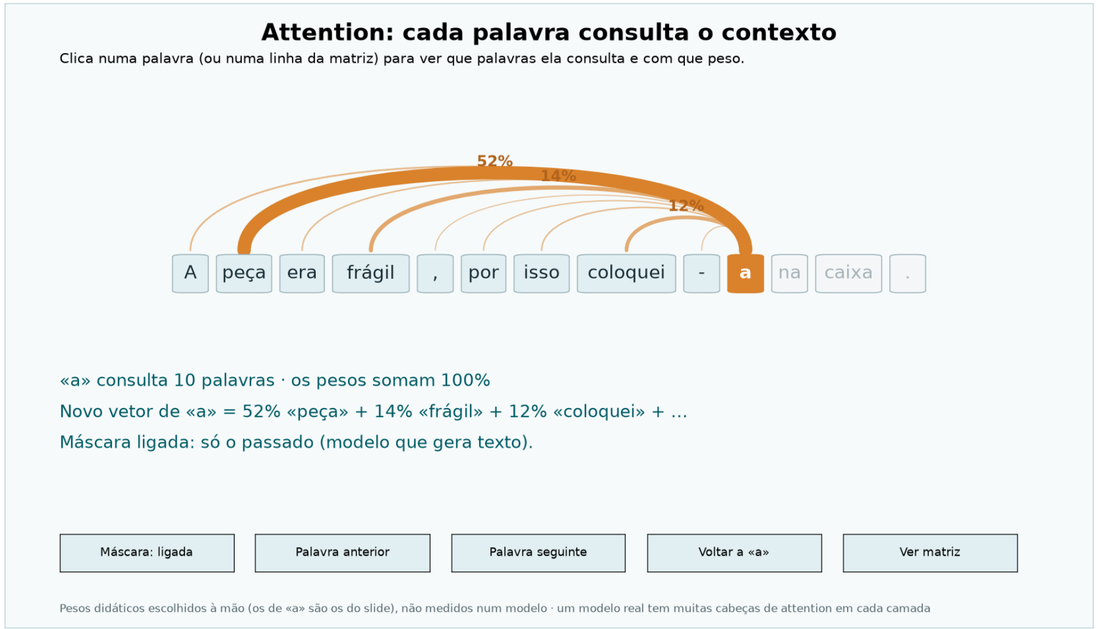
PROCESSAR: a atenção combina contexto e a arquitetura define as consultas permitidas.

Demonstração · exercício 03

```
python 03_attention.py
```

O que observar

- Clicar numa palavra: arcos para as palavras consultadas.
- Ligar e desligar a máscara causal.
- Os pesos de cada palavra somam 100%.

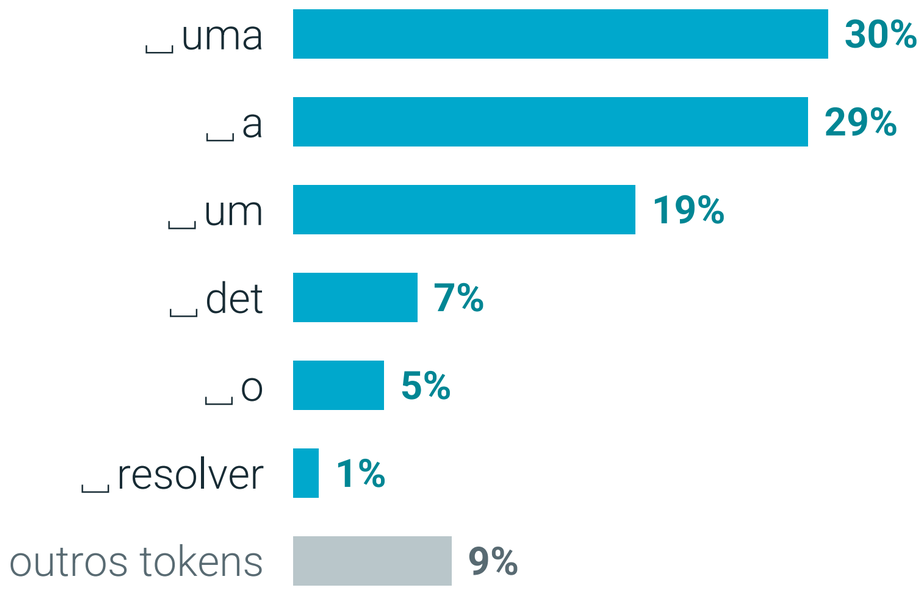


Captura do exercício: pesos didáticos, escolhidos à mão.

Prever o próximo token

GERAR: o decoder usa o contexto para prever e escolher o próximo token.

Contexto: «A produção foi retomada após …»



Tudo soma 100%.

Probabilidade alta não é verdade.

Valores medidos no AMALIA 9B (2026-10-12).

Geração autorregressiva: um token de cada vez

GERAR: o decoder usa o contexto para prever e escolher o próximo token.

Calcular probabilidades → Escolher um token → Acrescentar ao contexto → Repetir

O AMALIA, a escolher sempre o mais provável:

	1	2	3	4	5	6	7	8	9
A produção foi retomada após	**␣uma**	**␣fal**	**ha**	**␣de**	**␣energia**	**␣tempor**	**ária**	**.**	**fim**
	30%	28%	100%	22%	39%	34%	100%	28%	

Cada escolha entra no contexto da previsão seguinte, até ao token de fim.

Demonstração: próximo token

GERAR: o decoder usa o contexto para prever e escolher o próximo token.

Demonstração · exercício 04

```
python 04_proximo_token.py
```

O que observar

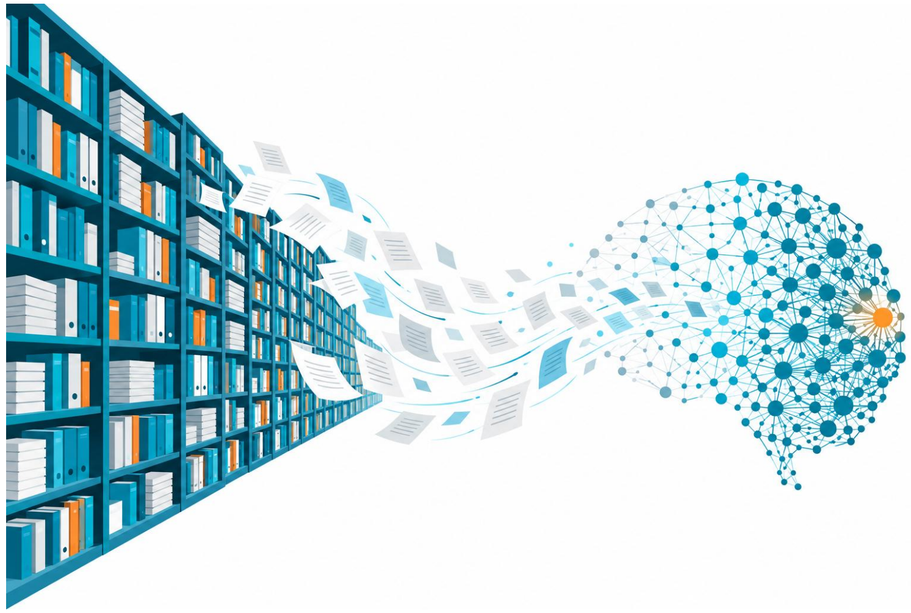
- «Mais provável» gera sempre a mesma frase.
- «Amostrar» pode seguir outro caminho.
- Palavras partidas em peças: ␣fal + ha.



Captura do exercício (respostas gravadas dos modelos).

Pré-treino: aprender a prever texto

TREINAR E ADAPTAR: usamos dados para aprender ou ajustar parâmetros.



O próprio texto dá as respostas

A	**linha**
A linha	**2**
A linha 2	**parou**
A linha 2 parou	**às**

Prever o seguinte, milhares de milhões de vezes: a perda ajusta os pesos.

Transfer learning: reutilizar aprendizagem

TRANSFERÊNCIA: reutilizar o que já foi aprendido, com ou sem novo treino.



Conhecimento de um modelo pré-treinado

↓ ↓

Reutilizar vetores
Ex.: pesquisa
por semelhança

Adaptar parâmetros
Ex.: fine-tuning

↓ ↓

Usar o aprendido numa nova tarefa

Transfer learning é a ideia geral. Fine-tuning é uma das formas de a aplicar.

Fine-tuning: adaptar os parâmetros

TREINAR E ADAPTAR: usamos dados para aprender ou ajustar parâmetros.



Ponto de partida: modelo pré-treinado

↓

Continuar o treino com exemplos da tarefa

Sensor avariado	Peça com fissura	Entrega atrasada
Manutenção	**Qualidade**	**Logística**

↓

Modelo adaptado: classifica ocorrências

Fine-tuning é uma forma de transfer learning. O treino adicional atualiza parâmetros.

Instruction tuning: um tipo de fine-tuning

TREINAR E ADAPTAR: usamos dados para aprender ou ajustar parâmetros.

Instrução	Resposta desejada
Resume a ocorrência numa frase. →	A linha 2 parou por uma falha no sensor e retomou após a sua substituição.
Devolve apenas a categoria da ocorrência. →	Manutenção

Instruction tuning é fine-tuning com pares instrução–resposta. Os parâmetros mudam.

RLHF e RLAIF: aprender com preferências

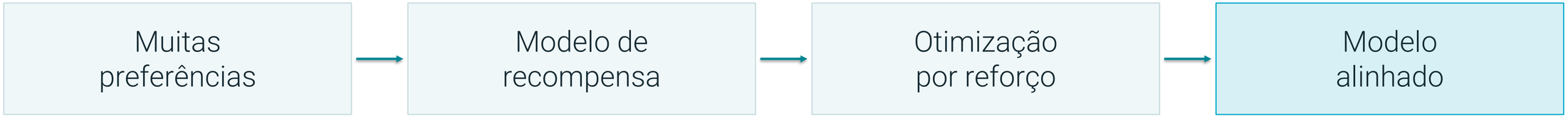
TREINAR E ADAPTAR: usamos dados para aprender ou ajustar parâmetros.

Duas respostas à mesma pergunta: qual é a melhor?

preferida

A: A linha 2 parou por uma falha no sensor.

B: A linha 2 parou por falta de manutenção preventiva.

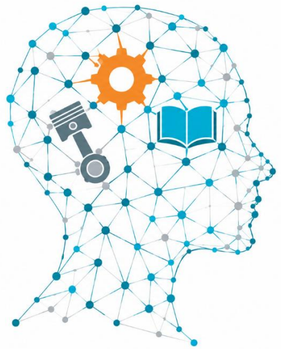


Feedback humano (RLHF) ou de IA (RLAIF) orienta preferências.
O treino atualiza parâmetros para favorecer respostas preferidas.

Usar o modelo: contexto e parâmetros fixos

UTILIZAR: o modelo já está treinado e os parâmetros mantêm-se fixos.

Nos parâmetros



Aprendidos no treino

Fixos durante a inferência

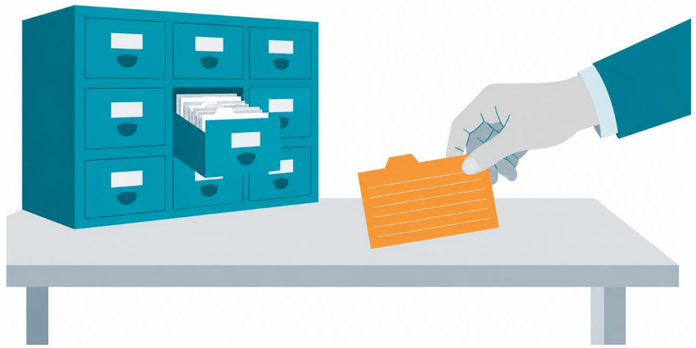
No contexto



Disponível nesta chamada

Instruções, exemplos e dados

Na memória da aplicação



Guardada pela aplicação

Reenviada como contexto

Dar informação no pedido orienta a resposta. Os parâmetros mantêm-se fixos.

Janela de contexto: o que cabe numa chamada

UTILIZAR: o modelo já está treinado e os parâmetros mantêm-se fixos.

Uma chamada com instrução, exemplos e ocorrência: 170 tokens medidos no AMALIA



Configuração local do AMALIA: janela de 32 768 tokens

este pedido ocupa 0,5%

Tudo tem de caber: instruções, histórico, documentos e resposta.

Few-shot prompting

UTILIZAR: exemplos no contexto, com os parâmetros do modelo fixos.

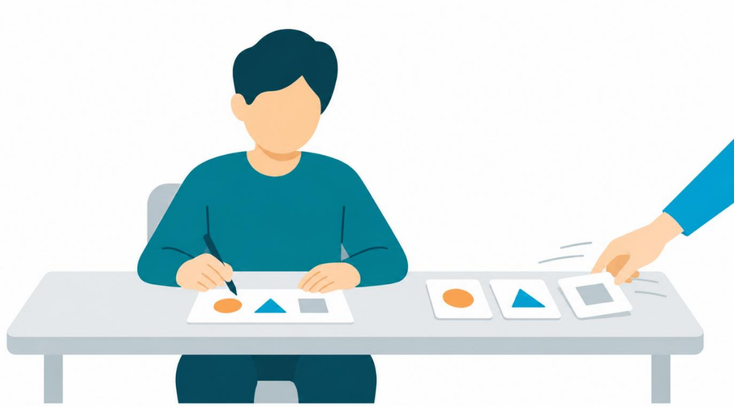
Alguns exemplos no pedido mostram o padrão da resposta.

Exemplos no pedido

Sensor avariado: Manutenção
Peça com fissura: Qualidade
Entrega atrasada: Logística

Classifica: «Rolamento bloqueado»

Resposta orientada pelos exemplos



Resposta ilustrativa: Manutenção.
Sem novo treino ou atualização dos parâmetros.

Zero-shot, one-shot e few-shot no pedido

UTILIZAR: o modelo já está treinado e os parâmetros mantêm-se fixos.

Caso a classificar

«Sensor sem sinal após impacto. A peça apresenta fissura.»

Pedido

Classifica em Manutenção, Qualidade ou Logística
+ 0, 1 ou 3 exemplos

Resposta do AMALIA

Sem exemplos	**Manutenção**
Um exemplo	**Manutenção**
3 exemplos	**Qualidade**
Exemplos + regra	**Qualidade**

Os exemplos mudaram a resposta.

Uma execução por condição, temperatura 0: não prova uma melhoria geral.

Hiperparâmetros de treino e controlos de geração

UTILIZAR: o modelo já está treinado e os parâmetros mantêm-se fixos.

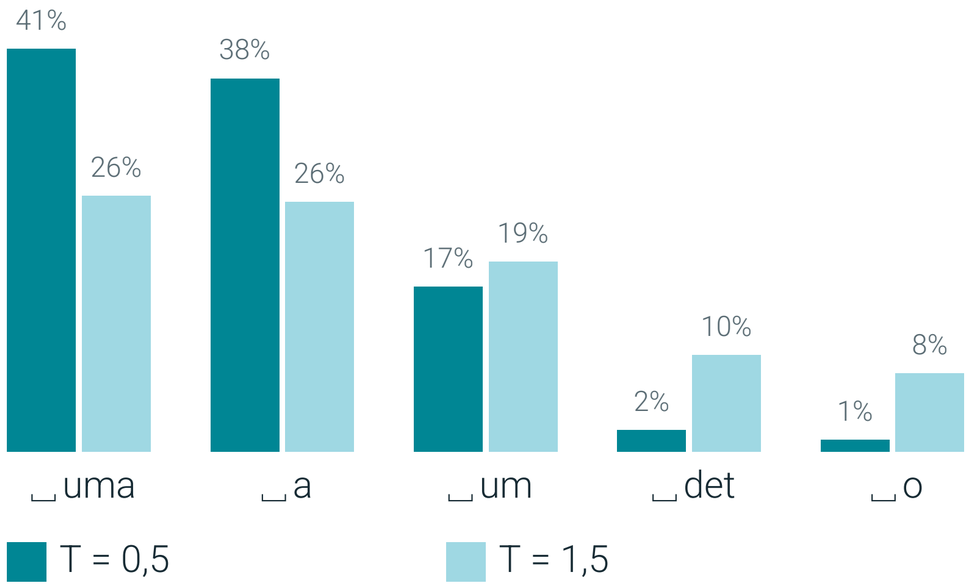
De treino	De geração
taxa de aprendizagem épocas tamanho do lote	temperatura top-p máximo de tokens
regulam a atualização dos parâmetros	**regulam a escolha e a paragem**

Na geração, temperatura, top-p e limites não atualizam os parâmetros.

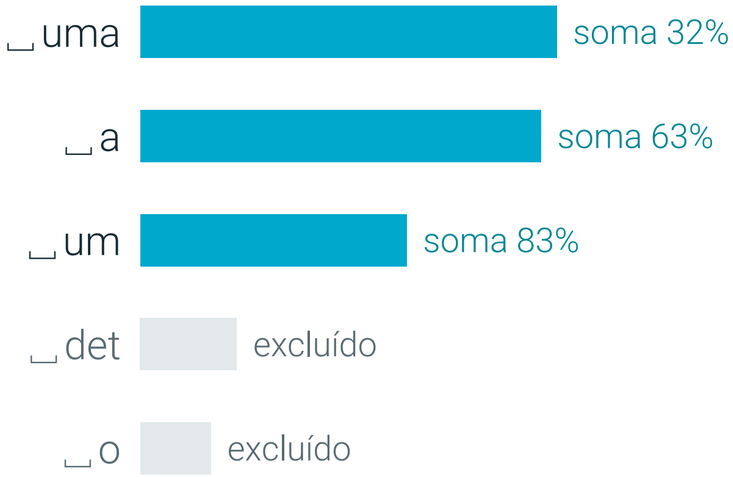
Temperatura e top-p

UTILIZAR: o modelo já está treinado e os parâmetros mantêm-se fixos.

Temperatura: concentra ou espalha



Top-p = 0,8: até somar 80%



Distribuição real do AMALIA para «A produção foi retomada após» (10 candidatos, 93%).

Demonstração: temperatura e top-p

UTILIZAR: o modelo já está treinado e os parâmetros mantêm-se fixos.

Demonstração · exercício 04

```
python 04_proximo_token.py
```

O que observar

- Temperatura baixa: a barra maior cresce.
- Top-p: os candidatos excluídos ficam a cinzento.
- Amostrar várias vezes com a mesma configuração.



Captura do exercício (respostas gravadas dos modelos).

Quando para a geração

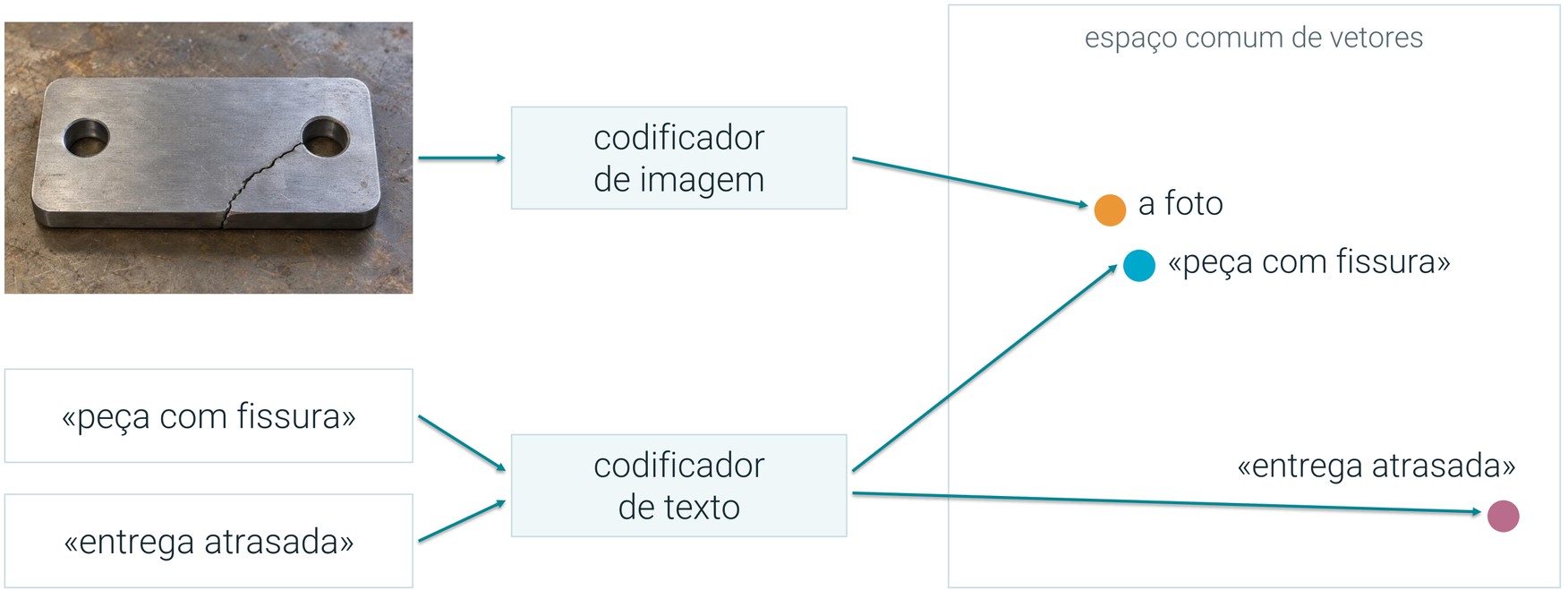
UTILIZAR: o modelo já está treinado e os parâmetros mantêm-se fixos.

Token de fim completa	«A linha 2 parou às 10h15 devido a uma falha no sensor e retomou a produção às 10h35 após a sua substituição.»
Limite: max_tokens = 8 cortada	«A linha 2 parou às …»
Sequência de paragem parou na 1.ª linha	«- Linha: 2»

Respostas reais do AMALIA 9B, temperatura 0 (2026-10-12).

CLIP: alinhar representações de imagem e texto

ALARGAR A ENTRADA: também podemos representar imagens por vetores.



CLIP compara imagem e texto. O modelo de linguagem gera a resposta.

Demonstração: imagem e texto

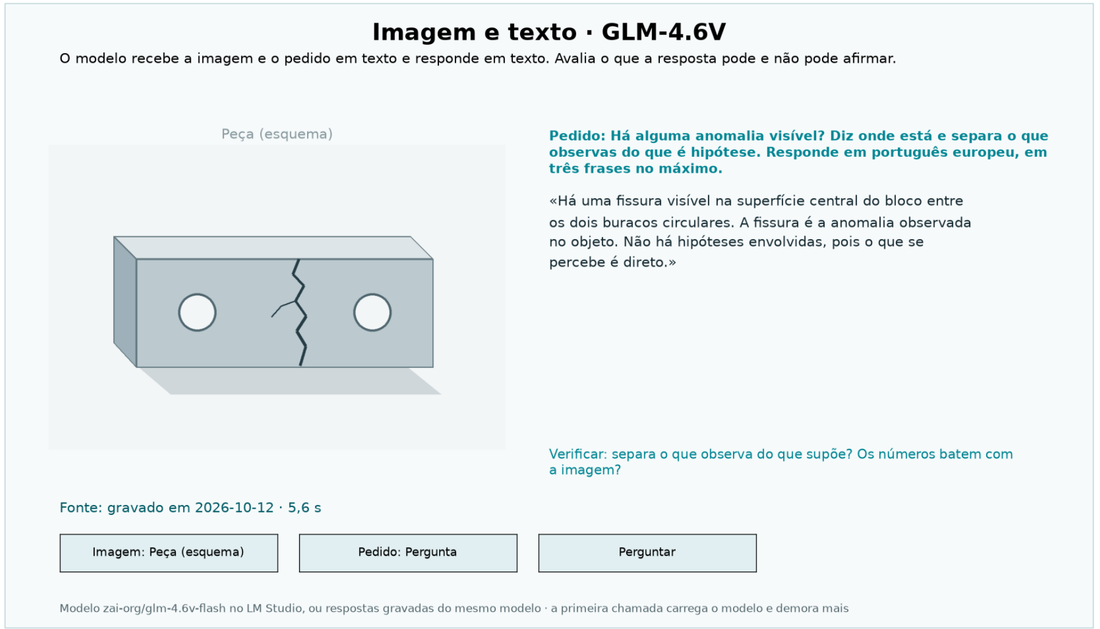
ALARGAR A ENTRADA: também podemos representar imagens por vetores.

Demonstração · exercício 06

```
python 06_imagem_texto.py
```

O que observar

- Descrever: corresponde ao que se vê?
- Pergunta: separa o que observa do que supõe?
- Limites: não inventa material, causa nem gravidade.



Captura do exercício (respostas gravadas dos modelos).

Pausa para café

